AI-BASED OCCUPANCY DETECTION FOR INTELLIGENT ENERGY SAVING AND APPLIANCE CONTROL

A PROJECT REPORT

EE19811 PROJECT WORK / PHASE II

Submitted by

SAIRISI N B (220901088)

SRI MAANESH PA (220901106)

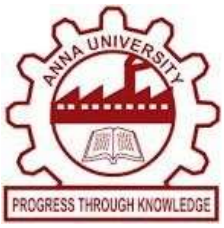
in partial fulfillment for the award of the degree

of

BACHELOR OF ENGINEERING

in

ELECTRICAL AND ELECTRONICS ENGINEERING



RAJALAKSHMI ENGINEERING COLLEGE

THANDALAM, CHENNAI – 602 105

ANNA UNIVERSITY: CHENNAI – 600 025

APRIL 2026

BONAFIDE CERTIFICATE

Certified that this report **"AI-BASED OCCUPANCY DETECTION FOR INTELLIGENT ENERGY SAVING AND APPLIANCE CONTROL"** is the bonafide work of **SAIRISI N B (2116220901088) and SRI MAANESH PA (2116220901106)** who carried out the project under my supervision. Certified further that to the best of my knowledge the work reported herein does not form part of any other thesis or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidate.

This project supports **SDG 7 – Affordable and Clean Energy** and **SDG 9 – Industry, Innovation and Infrastructure** by enabling AI-driven energy savings through intelligent appliance control, and promoting smart building automation using modern embedded and edge-computing technologies.

Dr. M. SRIDEVI,
HEAD OF THE DEPARTMENT
Professor,
Department of Electrical and Electronics Engineering,
Rajalakshmi Engineering College,
Thandalam, Chennai- 602 105.

Dr. V.GEETHA PRIYA,
SUPERVISOR
Professor,
Department of Electrical and Electronics Engineering,
Rajalakshmi Engineering College,
Thandalam, Chennai- 602 105.

Submitted for the exam held on _____

Internal Examiner **External Examiner**

PROJECT OUTCOME MAPPING

PO No.	Program Outcome	Relevance to our AI-Based Occupancy Detection Project	Level of Attainment
PO1	Engineering Knowledge	Applied AI-based detection and embedded control using Raspberry Pi to manage energy-efficient appliances.	3
PO2	Problem Analysis	Analyzed excess power usage in indoor spaces and provided a zone-wise occupancy solution to avoid wastage.	3
PO3	Design/Development of Solutions	Developed an automatic load control system using YOLOv8 detection and relay/MOSFET switching for appliances.	3
PO4	Conduct Investigations of Complex Problems	Evaluated detection accuracy using precision, recall, and F1-score and validated logic through MATLAB.	3
PO5	Engineering Tool Usage	Used YOLOv8, Python, Jupyter Notebook, MATLAB/Stateflow, Raspberry Pi for detection, analysis, and automation control.	3
PO6	Engineering and World	Supports smart energy usage in buildings like classrooms, offices, and auditoriums to benefit society.	2
PO7	Ethics	Ensured privacy-friendly operation by processing images locally on edge devices without cloud dependency.	2
PO8	Individual and Collabrative Work	Coordinated tasks in data collection, training, coding, and hardware testing with effective teamwork.	2
PO9	Communication	Demonstrated results through diagrams, simulations, and well-structured report	3
PO10	Project Management & Finance	Effectively managed project planning, resources, and cost using economical components and proper task coordinatin	3
PO11	Life-long Learning	Continously learned and applied new technologies like YOLO, Python, and embedded systems to solve real-world problems.	3

(Low-1, Medium-2, High-3)

ABSTRACT

In modern indoor environments such as classrooms, auditoriums, and office spaces, electrical appliances like lights, fans, and air conditioners are often operated without considering actual occupancy, leading to significant energy wastage and increased operational costs. To address this issue, this project proposes an AI-Based Occupancy Detection System for Intelligent Energy Saving and Appliance Control, which enables dynamic control of electrical loads based on real-time human presence. The proposed system utilizes a Raspberry Pi 4 integrated with a camera module to continuously capture images of the indoor environment. These images are processed using a trained YOLOv8 object detection model to accurately detect and count occupants. The indoor area is divided into multiple virtual zones, allowing the system to perform zone-wise occupancy analysis. Based on the detected occupancy in each zone, control signals are generated through the Raspberry Pi's GPIO pins to operate relay which in turn control electrical appliances such as lights and fans. The project is implemented in two phases. Phase I focuses on software development, including dataset collection, annotation, model training using YOLOv8, and performance evaluation using metrics such as precision, recall, F1-score, and mAP. Phase II focuses on hardware implementation, where the trained model is deployed on the Raspberry Pi to achieve real-time automation of appliances. The system demonstrates high detection accuracy and efficient response in controlling appliances based on occupancy, thereby reducing unnecessary energy consumption. It provides a scalable, cost-effective, and intelligent solution suitable for smart buildings and energy-efficient infrastructures. Overall, the proposed system contributes to sustainable energy management and supports the development of smart and automated environments.

ACKNOWLEDGEMENT

We wish to express our deep sense of gratitude to our guide **Dr.V.GEETHA PRIYA**, Professor, Department of Electrical and Electronics Engineering for providing an opportunity to work on this project **"AI BASED OCCUPANCY DETECTION FOR INTELLIGENT ENERGY SAVING AND APPLIANCE CONTROL"** under her valuable guidance. We also thank her for her encouragement in completing the project.

We express our sincere thanks to **Dr. M. SRIDEVI**, Professor and Head of the Department for his guidance and encouragement during this project.

We extend our thanks to **Dr. T. S. SARAVANAN** project coordinator**,** Assistant Professor(SG), Department of Electrical and Electronics Engineering, for his support.

We extend our thanks to **Dr. M. SUBBIAH**, Emeritus Professor of Department of Electrical and Electronics Engineering, for his guidance during this project.

We express our thanks to **Dr. S.N. MURUGESAN**, Principal, Rajalakshmi Engineering College, and the management for their kind support and the facilities provided to complete this work in time.

TABLE OF CONTENTS

LIST OF TABLES

LIST OF FIGURES

LIST OF ABBREVIATIONS AND SYMBOLS

YOLO	You Only Look Once
CNN	Convolutional Neural Network
BLE	Bluetooth Low Energy
ELM	Extreme Learning Machine
CNN	Convolutional Neutral Network
LSTM	Long Short-Term Memory
FPS	Frames Per Second
mAP	Mean Average Precision
TP	True Positive
TN	True Negative
FP	False Positive
FN	False Negative
RPI	Raspberry Pi
GUI	Graphical User Interface
PIR	Passive Infrared Sensor
PSU	Power Suppy Unit
PWM	Pulse Width Modulation
HVAC	Heating, Ventilation and Air Conditioning
GMM	Gaussian Mixture Model

DSC	Decision Tree Sensor Classification
MOSFET	Metal Oxide Semiconductor Field Effect Transistor
SIM	Stateflow/Simulink Model

CHAPTER 1

INTRODUCTION

1.1 GENERAL

In large indoor environments such as auditoriums, classrooms, and offices, electrical appliances including lights, fans, and air conditioners are often left operating even in the absence of occupants, leading to significant energy wastage and higher operational costs. To overcome this issue, the proposed system virtually divides the indoor area into multiple zones or grids, where a Raspberry Pi 4 integrated with a camera module continuously monitors occupancy in each zone. Using the YOLOv8 object detection model, the system accurately detects and counts people in real time, enabling intelligent zone-wise control of appliances through relays or MOSFET driver circuits. Electrical loads in occupied zones are automatically switched ON, while appliances in unoccupied areas are turned OFF, thereby ensuring optimal energy consumption and automation efficiency. The system is designed for easy deployment in various environments and minimizes manual intervention, making it suitable for smart building applications. Furthermore, it can be extended with mobile or web dashboards for monitoring and logging occupancy data and energy savings over time. Its scalability allows adaptation to both small rooms and large halls by adjusting the number of zones and camera placements. By reducing unnecessary power usage, the system effectively lowers energy bills and provides a rapid return on investment compared to conventional automation systems.

1.2 LITERATURE SURVEY

1. Aftab *et al*. "Automatic HVAC Control with Real-Time Occupancy Recognition." – 2017. Proposed an automatic HVAC control system using a Raspberry Pi 3 with video-based occupancy recognition and model predictive control. The system successfully optimized HVAC usage based on real-time occupancy detection, reducing unnecessary energy consumption. However, the implementation lacked zone-wise automation with relays, limiting its effectiveness in managing appliances at a finer level. This study proposes a two-layer hierarchical control scheme: a primary layer using adaptive droop control for load sharing and a secondary layer for DC bus voltage restoration. Their method, tested in simulation, improves current sharing and voltage regulation by 9.65% and 6.67%, respectively— demonstrating how adaptive droop control can enhance the reliability of microgrids with multiple DERs.

2. Kumar *et al*. (2021) BLE-Based Occupancy. "Estimating Indoor Occupancy through Low Cost BLE Devices." – 2021. The BLE-Based Occupancy system proposed in 2021 utilized Bluetooth Low Energy (BLE) devices to estimate indoor occupancy levels. BLE works by transmitting and receiving signal variations from devices within a space, which are then processed to estimate the number of people present. This approach provided a low-cost and privacy-friendly alternative to camera-based methods, since no visual data is required. The system demonstrated an accuracy of nearly 98%, proving its effectiveness in detecting occupancy. However, the limitation of this method lies in its inability to provide zonal or grid-based monitoring. It could only give a whole-room occupancy count, without indicating where exactly people were located inside the environment. This restricts its application in smart energy management systems, where zone-wise control of appliances such as lights, fans, and motors is essential to minimize energy wastage.

3. Krati Rastogi and Divya Lohani, "IoT-based Indoor Occupancy Estimation Using Edge Computing", Shiv Nadar University, Gautam Buddha Nagar, India, 2020.

   The study compares cloud and edge computing for IoT-based indoor occupancy estimation in classrooms. Using CO_2, RH, and temperature data. Occupancy estimation Model: Models were developed with quantile regression (QR) achieving higher accuracy (MAPE: 2.51%). Edge computing reduced latency and network traffic, proving more efficient for real-time occupancy monitoring.

4. Ramoni Adeogun, Ignacio Rodriguez, Mohammad Razzaghpour, Gilberto Berardinelli and Preben Elgaard Mogensen, ”Indoor Occupancy Detection and Estimation using Machine Learning and Measurements from an IoT LoRa based Monitoring System”, Wireless Communication Networks Section, Department of Electronic Systems, Aalborg University, Nokia Bell Labs, Aalborg, Denmark.

   The study explores machine learning for IoT-based occupancy detection in office spaces. Using environmental sensors (CO_2, humidity, motion, sound, light), a feedforward neural network (FNN) achieved 94.6% accuracy in presence detection and 91.5% in multi-class classification. Accuracy dropped in different rooms due to environmental variations. Future work includes network optimization and larger datasets.

5. Mustafa K. Masood and Victor W.-C. Chang, "Real-time occupancy estimation using environmental parameters", Yeng Chai Soh School of Electrical and Electronic Engineering Nanyang Technological University Singapore. This paper proposed a real-time occupancy estimation system designed to optimize Air Conditioning and Mechanical Ventilation (ACMV) for better energy efficiency in buildings. Instead of relying on cameras or motion sensors, the system collected environmental parameters such as CO_2 concentration, temperature, humidity, and light levels, which correlate strongly with human presence and activity.

To process these inputs, the authors implemented a wrapper-based feature selection model combined with an Extreme Learning Machine (ELM) algorithm. This approach allowed the system to automatically select the most relevant environmental features, ensuring higher accuracy while reducing computational complexity compared to conventional methods. The results demonstrated that this method could provide accurate occupancy estimates in real time, making it suitable for energy optimization in HVAC systems. However, the approach was still limited in providing spatial resolution (zone-wise occupancy), which is important for fine-grained load control.

6. Ebenezer Hailemariam, Rhys Goldstein, Ramtin Attar and Azam Khan, "Real-Time Occupancy Detection using Decision Trees with Multiple Sensor Types", Autodesk Research, Toronto, Ontario, Canada. The paper introduces a real-time occupancy detection system designed to optimize HVAC operations and improve space utilization in indoor environments. The system relies on low-cost environmental sensors such as CO_2 concentration, temperature, humidity, and motion, which provide indirect indicators of human presence. To process this data, the authors employed a Decision Tree classification algorithm, which offered a balance between simplicity, interpretability, and reasonable accuracy. The results showed that the Decision Tree was effective in detecting occupancy trends and supporting automated building energy management. Despite these limitations, the work demonstrated that sensor-based machine learning approaches can serve as a cost-effective and scalable solution for occupancy monitoring in smart buildings, particularly when combined with more robust algorithms or hybrid sensor modalities.
7. J. Ahmad, H. Larijani, R. Emmanuel and M. Mannion, “Occupancy detection in non-residential buildings – A survey and novel privacy preserved occupancy monitoring solution”, School of Computing, Engineering and Built Environment, Glasgow Caledonian University, Glasgow, United Kingdom. The system employed

Gaussian Mixture Models (GMMs) for background subtraction and morphological operations for refining object detection, enabling reliable occupancy monitoring in real time. By combining detection algorithms with encryption, the solution maintained high detection accuracy while ensuring that sensitive video data was protected. This work highlighted the importance of balancing accuracy, privacy, and security in occupancy monitoring systems, making it particularly relevant for buildings such as offices, schools, and public facilities. However, it did not extend to zonal or grid-based detection and control, which limits its direct application in energy-optimized load management.

8. Erickson *et al*. Energy Efficient Building Environment Control Strategies Using Real-Time Occupancy Measurements, 2009. This study introduced a wireless camera-based occupancy monitoring system designed to improve building energy efficiency by dynamically controlling HVAC operations. The system continuously monitored real-time occupancy levels and adjusted the HVAC settings according to room usage, thereby preventing unnecessary energy consumption during unoccupied periods. Through this adaptive control strategy, the researchers reported an approximate 14% reduction in overall energy usage compared to conventional fixed-schedule systems. The approach demonstrated the potential of integrating visual sensing with automation for energy savings. However, the system operated only at the whole-room level and did not support zone-specific control, limiting its precision and scalability for larger or multi-zone environments. The research laid the groundwork for further studies integrating AI and IoT technologies for fine-grained, zone-wise load management. However, it lacked zone-level automation, reducing its effectiveness in large spaces.

9. Matuška *et al*. (2022) "An Improved IoT-Based System for Detecting the Number of People and Their Distribution in a Classroom", Matuška et al. proposed an improved

IoT-based system designed specifically for monitoring occupancy and seat distribution in classroom environments. The system employed ultrasonic distance sensors positioned at entry points and on desks to track the presence of individuals. A Raspberry Pi was used as the central controller to process the sensor data in real time. Additionally, IoT nodes deployed across seating areas mapped which desks were occupied, enabling both seat-level occupancy detection and distribution visualization. However, while effective for classroom settings, the reliance on ultrasonic sensors limited its accuracy in large, dynamic spaces compared to AI-based image processing methods.

10. Tan *et al*. (2022) "Real-Time Occupancy Estimation Using Smartphone Data and Deep Learning." – IEEE Transactions on Mobile Computing, 2022. Analyzed Tan et al. (2022) presented a novel method for real-time indoor occupancy estimation by leveraging data collected from users' smartphones. The study utilized anonymized sensor logs such as accelerometer readings, Wi-Fi signals, and Bluetooth activity to infer human presence without requiring additional hardware installations like cameras or PIR sensors. To process this data effectively, the authors implemented deep learning models, specifically Convolutional Neural Networks (CNN) and Long Short-Term Memory (LSTM) networks, which enabled both spatial and temporal analysis of occupancy patterns. The system was tested in office environments and successfully classified occupancy status in real time, proving useful for applications such as smart HVAC scheduling and space management. The approach offered advantages in terms of privacy preservation (no visual data captured) and cost-effectiveness since it relied on existing smartphones. However, its dependency on users carrying smartphones and connectivity variations limited its reliability in all scenarios compared to dedicated vision-based systems.

1.3 SUMMARY OF LITERATURE SURVEY

To address these issues, later studies shifted towards sensor- and IoT-based approaches. For example, Hailemariam et al. (2016), Adeogun et al. (2020), and Masood & Chang (2020) utilized environmental parameters (CO_2, temperature, humidity, motion, sound, light) with machine learning techniques like Decision Trees, Neural Networks, and Extreme Learning Machines, achieving accuracies above 90% but encountering challenges from sensor anomalies and environmental variations.

Research highlights the importance of edge computing in reducing latency and dependency on cloud servers, thereby ensuring faster decision-making and enhanced data privacy in smart building management. Recent studies emphasize energy savings of 20–30% when integrating accurate occupancy detection with HVAC control, underscoring the practical benefits and sustainability impact of these systems. Despite advancements, challenges remain in scalability, interoperability, sensor calibration, and real-world deployment, highlighting the need for standardized frameworks and protocols for widespread adoption.

Overall, the literature shows a clear progression from camera-based systems toward IoT-, sensor-, and ML-enabled solutions, balancing accuracy, privacy, cost-effectiveness, and scalability. The consensus is that future systems will likely rely on hybrid approaches, combining multiple sensor types, edge intelligence, and privacy-preserving mechanisms to enable robust and efficient occupancy-driven energy management in smart buildings.

1.4 OBJECTIVES

PHASE I

- To design a complete zonal control system using MATLAB Stateflow, enabling real-time decision-making based on occupancy transitions.
- To implement AI-based object detection and accurate head counting using a YOLO-trained model for reliable indoor sensing.
- To train the model with improved datasets and send detected count directly to end users/controllers.

PHASE II

- To implement zone-wise appliance management using relays or MOSFETs, ensuring appliances operate only where required.
- To optimize energy usage by controlling the appliances in each zone.
- To create a scalable and cost-effective solution suitable for smart buildings, reducing energy wastage and improving operational efficiency.

1.5 CHAPTER SUMMARY

This chapter presented the simulation and performance evaluation of the proposed AI-based occupancy detection system. The dataset preparation, training process, and model testing were discussed in detail. The YOLO-based detection model was validated using different occupancy conditions. Performance metrics were analyzed to evaluate detection accuracy and reliability. The simulation results confirmed the effectiveness of the proposed system before hardware implementation.

CHAPTER 1

Introduces the AI-based occupancy detection system, problem of energy wastage, literature review, and project objectives.

CHAPTER 2

Describes the system architecture, components, and zone-wise automation for energy-efficient appliance control.

CHAPTER 3

Presents simulation using MATLAB and YOLO performance evaluation to validate system accuracy.

CHAPTER 4

Explains hardware implementation using Raspberry Pi and real-time zone-based appliance control.

CHAPTER 5

Concludes the project outcomes and suggests future improvements like IoT integration and scalability.

1.6 CHAPTER CONCLUSION

This chapter provides a comprehensive overview of the AI-based occupancy detection system for intelligent energy saving and appliance control, highlighting the need for efficient energy management in indoor environments where traditional systems fail due to the absence of real-time occupancy awareness. It explains how the integration of Raspberry Pi, camera-based monitoring, and AI-driven detection using deep learning models enables accurate and reliable human detection, overcoming the limitations of conventional sensor-based approaches. The literature survey emphasizes the evolution from basic sensors to IoT- and AI-enabled systems, while identifying key gaps such as lack of zone-wise control, scalability, and adaptability. To address these challenges, the proposed system adopts a zone-based detection

approach using a YOLO model, allowing precise control of electrical loads and improved energy efficiency. The incorporation of IoT facilitates real-time monitoring through a mobile application, while edge computing ensures faster processing and reduced dependency on internet connectivity. Overall, this chapter establishes a strong foundation by justifying the need for the proposed solution and sets the stage for the detailed system design, implementation, and evaluation discussed in the subsequent chapters.

CHAPTER 2

OCCUPANCY DETECTION AND CONTROL

2.1 INTRODUCTION

This chapter focuses on the design and working principle of the proposed AI-Based Occupancy Detection for Intelligent Energy Saving and Appliance Control. The system aims to minimize energy wastage in indoor environments by automatically controlling electrical appliances based on real-time occupancy detection. The chapter presents the overall system architecture through a block diagram, describing how various hardware and software components interact to achieve intelligent automation. Each block — including the camera module, Raspberry Pi, YOLOv8 object detection, and relay/MOSFET driver — is explained to illustrate its function in the process. The integration of AI-based detection with IoT-enabled ensures optimal energy utilization in smart building applications.

2.2 FUNCTIONAL BLOCK DIAGRAM

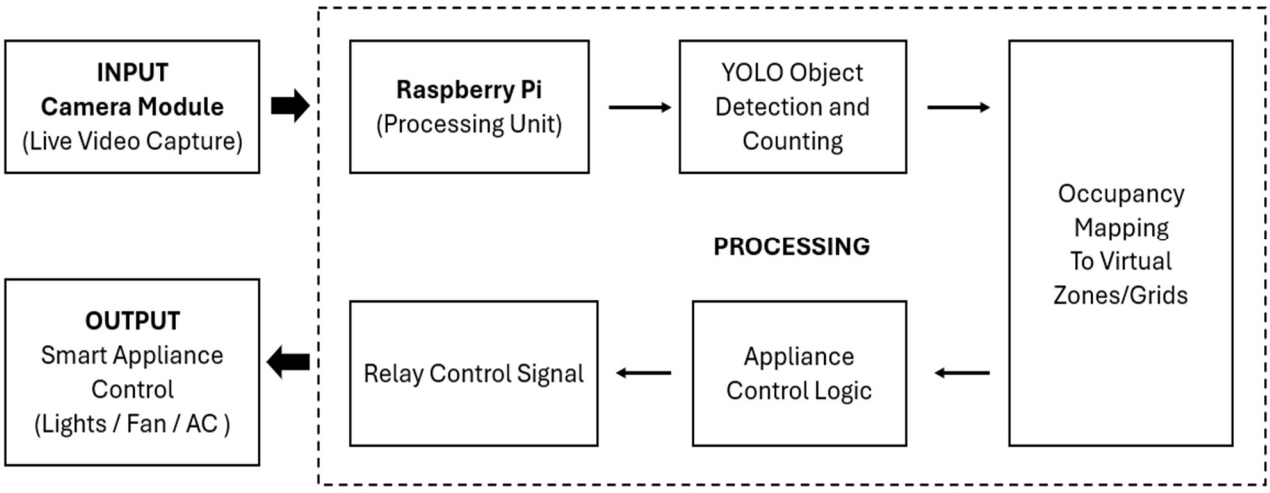


Fig 2.1 Block Diagram of Occupancy Detection Control

Fig 2.1 shows that the system starts with a Raspberry Pi Camera Module, which continuously captures real-time images of the room. These images are sent to the Raspberry Pi 4, which acts as the central processing unit. Inside the Raspberry Pi, a trained YOLOv8 Object Detection Model identifies and counts humans present in the frame. Based on the detected positions, the system assigns each person to a virtual zone within the indoor space. A decision-making logic then checks which zones are occupied. If a zone contains one or more people, the system activates the appliances (such as lights, fans, or AC units) in that zone. If the zone is empty, the appliances remain OFF, thereby avoiding unnecessary power usage. The control signals generated by the Raspberry Pi are sent to the appliances through a Relay or MOSFET Driver Circuit, ensuring safe switching of electrical loads. This setup creates a closed-loop automation system, where real-time occupancy directly controls energy consumption without any manual intervention, supporting intelligent, scalable, and sustainable operation in smart buildings.

2.3 COMPONENTS OF BLOCK DIAGRAM

1. Camera Module (Input Model)

The camera module acts as the input model of the system. It captures real-time images or video frames of the monitored indoor environment. These frames serve as raw input data for occupancy detection. A Raspberry Pi camera module or USB webcam can be used depending on availability. The captured frames are continuously forwarded to the Raspberry Pi for further processing.

2. Raspberry Pi (Processing Model)

The Raspberry Pi functions as the processing model of the system. It receives the input frames from the camera module and performs AI-based computation. The Raspberry Pi runs the trained YOLOv8 model for object detection and executes the

control logic. It also generates output signals for appliance control through GPIO pins. This block acts as the brain of the entire system.

3. YOLO Object Detection and Counting Model

This block represents the AI detection model used in the system. The YOLOv8 algorithm processes the input images and detects occupants in real time. Bounding boxes are generated around detected objects along with confidence scores. The number of detected occupants is counted and passed to the next stage. This model ensures accurate detection and fast processing for real-time automation.

4. Occupancy Mapping to Virtual Zones Model

The occupancy mapping model divides the monitored area into multiple virtual zones. The detected occupants are assigned to their respective zones based on position coordinates. Each zone corresponds to a specific electrical load. This mapping helps identify which area is occupied and enables zone-wise control. The mapped output is sent to the control logic stage.

5. Appliance Control Logic Model

The appliance control logic model determines the ON/OFF status of appliances. If occupancy is detected in a particular zone, the corresponding appliance is activated. If no occupants are present, the appliance remains OFF. This conditional logic helps reduce unnecessary energy consumption. The decision output is sent to the relay driver circuit.

6. Relay Model

The relay model acts as an interface between the Raspberry Pi and electrical loads. The Raspberry Pi sends low-voltage control signals to the relay module. The relay switches high-voltage appliances based on these signals. This block ensures

safe isolation and reliable switching. It enables real-time control of lights, fans, and other appliances.

7. Smart Appliance Control (Output Model)

The output model controls the electrical appliances in each zone. Lights, fans, or AC units are automatically turned ON or OFF based on occupancy. This stage performs intelligent energy management. The output reflects the final action of the system. The overall process results in automatic and energy-efficient appliance control.

2.4 CHAPTER SUMMARY

This chapter presented the detailed block diagram of the proposed system and explained the function of each component in achieving the overall objective of smart, zone-based load control. The system workflow begins with the camera module, which captures live video of the indoor space. The Raspberry Pi 4, serving as the central processing unit, executes the YOLOv8 detection algorithm to identify and count people within the frame. The detected occupants are then mapped to specific zones, enabling zonal-level control of loads. The decision-making logic embedded in the Raspberry Pi evaluates occupancy status and sends appropriate control signals to the relay or MOSFET driver circuit, which in turn switches electrical loads such as fans, lights, and motors ON or OFF. This closed-loop control system operates continuously, adapting in real time to changes in occupancy.

The chapter also highlighted the importance of each hardware and software block: The Camera Module acts as the system's eyes, providing real-time input. The Raspberry Pi functions as the brain, handling AI inference and control logic. Overall, this architecture ensures optimized energy utilization, reduced manual intervention, and enhanced operational efficiency, aligning with the goals of smart building energy management.

2.5 CHAPTER CONCLUSION

This chapter concluded the design and operational explanation of the proposed zonal occupancy-based smart load control system. The functional block diagram was used to depict the step-by-step workflow, showing how the system processes real-time camera input, performs AI-based detection, maps people to zones, and dynamically controls connected appliances.

By integrating AI, IoT, and embedded automation, the system provides a sustainable and intelligent solution for energy conservation. The design ensures reliability, accuracy, and real-time responsiveness, making it suitable for large indoor environments. Furthermore, the modular structure allows for easy customization and scalability, enabling adaptation to various infrastructures such as classrooms, auditoriums, and office spaces.

The proposed framework successfully bridges the gap between traditional automation and modern AI-based control, ensuring precise energy management while maintaining user comfort. The next chapter will discuss the software simulation, algorithm flow, and closed-loop operation, validating the system's functionality and performance through MATLAB/Simulink and real-time implementation results.

CHAPTER 3

SIMULATION PHASE

3.1 INTRODUCTION

It provides studying classrooms, auditoriums, and office spaces to understand the energy wastage caused by appliances such as fans, lights, and motors being left ON irrespective of occupancy. During this phase, we observed that existing systems mostly relied on manual switching or PIR sensors, which lacked precision and operated at a whole-room level rather than zone-wise. Our secondary research explored AI-based object detection techniques, particularly YOLOv8, for more accurate and real-time occupancy monitoring. We also studied how the Raspberry Pi could act as an edge device to process detections locally and trigger electrical loads via relays or MOSFETs for automation.

For primary research, we interacted with faculty members, students, and facility managers to identify issues in energy management in classrooms and labs. The feedback highlighted excessive power consumption, lack of automation, and the difficulty of ensuring appliances were switched off after usage. Respondents suggested that an AI-powered system capable of zone-based detection and load control would not only save energy but also reduce human effort and improve efficiency. These insights shaped the design of our system that integrates Raspberry Pi, Pi Camera, YOLOv8, and relay modules for zonal, real-time load management.

3.2 SIMULATION SOFTWARE

MATLAB serves as the primary simulation platform for modeling and analyzing the proposed AI based Occupancy Detection for Intelligent Energy Saving and Appliance Control. It provides a robust environment for integrating algorithms,

control logic, and system behavior before implementing the design in hardware. Using MATLAB and Simulink, the decision-making process of the system—based on occupancy detection and zonal load control—was simulated to validate its performance and reliability.

The simulation replicates the real-world scenario where occupancy data from the camera (represented as input signals) determines the ON/OFF state of appliances through logical conditions. The Stateflow tool in Simulink was used to design a state-based control system representing different zones and their respective load behaviors. This simulation helps visualize transitions between occupied and unoccupied states and how control signals are generated for each zone. By using MATLAB, various parameters such as response time, control delay, and logical accuracy were analyzed to ensure stable system operation. The simulation results confirmed that the system efficiently activates loads only in occupied zones while keeping unoccupied areas powered off, validating its effectiveness before real-time implementation.

3.3 STATEFLOW MODEL

Stateflow is a powerful simulation and modeling tool integrated within MATLAB/Simulink, designed for developing and analyzing event-driven systems based on state machines and flow charts. It allows users to model complex decision logic that depends on conditions, events, and time-based transitions. In this project, Stateflow is used to simulate the decision-making logic for zone-wise appliance control based on occupancy detection. Each zone in the indoor environment is represented as an independent state that transitions between “Occupied” and “Unoccupied” based on input signals. The Stateflow model enables real-time visualization of how appliances such as lights, fans, and motors respond dynamically to changes in occupancy. By simulating control behavior before hardware

implementation, it helps validate system performance, ensures logical correctness, and supports the design of an efficient, automated energy management system.

Furthermore, the simulation provides a clear understanding of how the control logic reacts to multiple occupancy changes simultaneously across zones. This helps in optimizing timing, transitions, and energy-saving strategies before deploying the system in real-world environment.

The flexibility of the tool enables future scalability — additional zones, sensors, or control features can be incorporated without altering the existing framework. Overall, the Stateflow simulation plays a crucial role in validating the reliability, adaptability, and intelligence of the proposed AI-based energy-saving system before its real-time implementation.

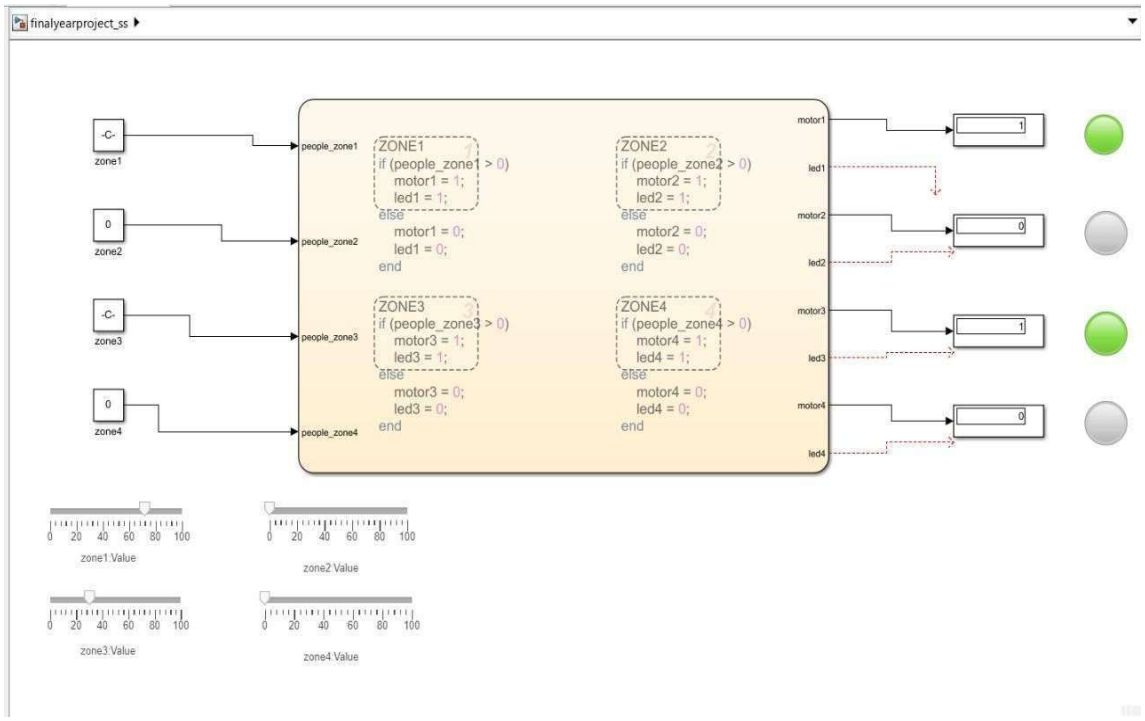


Fig. 3.1 Zone Detection Methodology using Stateflow

Fig. 3.1 shows that the Stateflow model developed for this project simulates the decision-making logic used in zone-based occupancy detection and appliance control. The model takes the number of people detected in each zone as input, obtained from the AI-based object detection system. The system is divided into four virtual zones—Zone 1, Zone 2, Zone 3, and Zone 4—and each zone has an independent control logic. The logic checks the occupancy status of each zone and activates or deactivates corresponding loads accordingly. If the number of people in a particular zone is greater than zero, the respective LED and motor (representing the light and fan or AC) are switched ON; otherwise, they remain OFF. The outputs are visually indicated through LEDs to represent the state of appliances in each zone, ensuring a clear and intuitive simulation of the system's operation.

This simulation model effectively demonstrates the closed-loop control process, where occupancy detection directly influences energy utilization. It allows real-time testing of logic conditions and verifies the accuracy of decision responses without requiring physical hardware at this stage. By analyzing the behavior of the Stateflow system, we ensure that the logic performs as intended for different occupancy conditions, supporting the transition to practical implementation using Raspberry Pi, relays, and MOSFETs. Thus, the Stateflow simulation acts as a critical design validation step before deploying the hardware-based intelligent energy control system.

3.4 DATA COLLECTION DIAGRAMS

To train the AI-based occupancy detection model, a custom dataset was created using a camera mounted above a zone-divided platform. The setup consisted of four virtual zones marked on the surface, with bulbs placed at each corner to represent zone-wise appliance control. Multiple objects representing occupants were placed randomly in different zones to simulate real-time occupancy conditions. Images

were captured under different lighting conditions, object positions, and varying numbers of occupants to improve model robustness.

Fig. 3.2 Zones Occupancy Sample Dataset

Fig. 3.2 shows a sample dataset image representing zone-based occupancy inside the experimental setup. The area is divided into four virtual zones, and a limited number of objects are placed in different regions to simulate human presence. Each object represents an occupant detected by the AI model. The variation in object placement helps the model learn spatial distribution and zone-wise mapping. These images were captured using the camera module and included in the dataset for training the occupancy detection system.

Approximately 1700 images were collected using the camera module, covering different scenarios such as single occupancy, multiple occupancy, empty zones, and partially occupied zones. The collected images were then uploaded to Roboflow for annotation. Bounding boxes were manually labeled around each object representing a detected occupant. After annotation, the dataset was automatically split into training, validation, and testing sets using a 70% training, 20% validation, and 10% testing ratio.

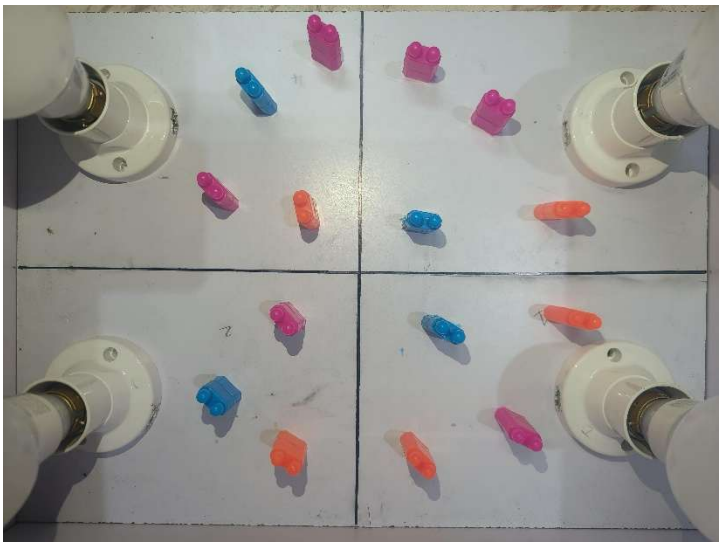


Fig. 3.3 Multiple Zones Occupancy Sample Dataset

Fig. 3.3 illustrates a dataset image with multiple occupants distributed across all zones. Several objects are placed randomly to simulate crowded indoor conditions and multi-zone occupancy. This type of dataset improves the model's ability to detect multiple people simultaneously and assign them to their respective zones. The images were collected under different configurations to increase dataset diversity. These samples help the YOLO-based model achieve accurate real-time occupancy detection for zone-wise appliance control.

The prepared dataset was then used to train the YOLO-based object detection model. This data collection process ensured that the model learns different occupancy patterns and improves detection accuracy for real-time zone-wise appliance control.

Additionally, the dataset includes variations in object position, orientation, and spacing to improve model robustness. Images were captured under different lighting conditions to reduce sensitivity to illumination changes. Multiple occupancy combinations were considered to represent real-time indoor scenarios. The dataset also contains partially occluded objects to train the model for complex environments. These variations help the model generalize better for unseen test

conditions. Overall, the collected dataset improves detection reliability and enhances zone-wise appliance control performance.

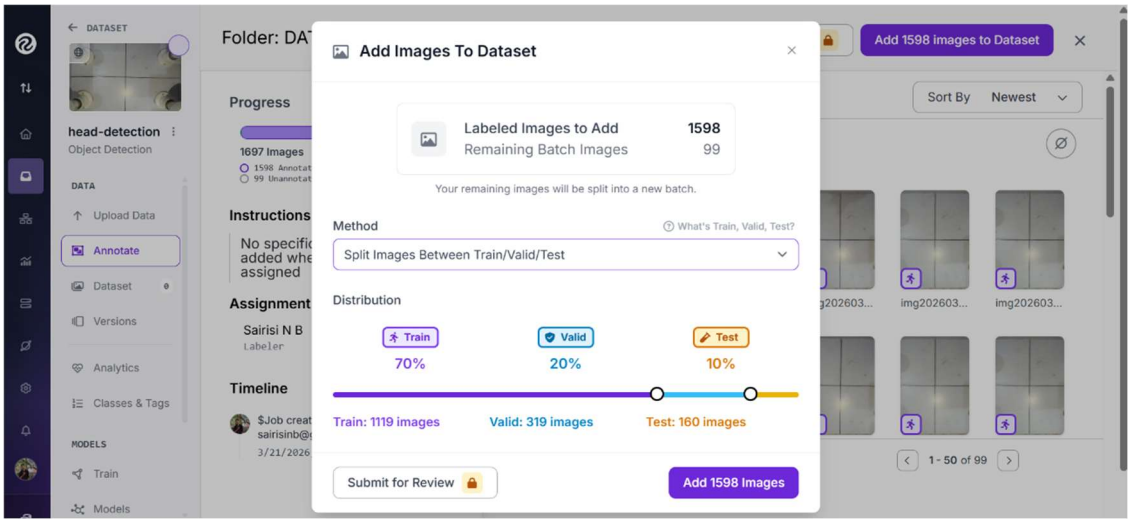


Fig. 3.4 Dataset Split Using Roboflow

Fig. 3.4 shows that the collected dataset was uploaded to the Roboflow platform for annotation and preprocessing. After labeling the objects, the dataset was automatically divided into training, validation, and testing sets. A split ratio of 70% for training, 20% for validation, and 10% for testing was used to ensure balanced model learning and evaluation. The training set contained 1119 images, the validation set included 319 images, and the testing set consisted of 160 images. This dataset distribution helps the YOLO model learn effectively, validate performance during training, and test accuracy on unseen data.

3.5 TRAINING DATASET

The training dataset consists of labeled images used to train the YOLO-based occupancy detection model. These images include different occupancy scenarios such as single-zone occupancy, multiple-zone occupancy, empty zones, and crowded conditions. The objects representing occupants were placed randomly in different zones to improve model generalization. A total of 70% of the dataset (1119

images) was used for training. During training, the model learns to detect objects and identify their positions within each zone. This dataset helps the model understand spatial distribution and improve detection accuracy for real-time implementation.

3.6 VALIDATION AND TESTING DATASET

The validation and testing datasets were used to evaluate the performance of the trained model. The validation dataset contained 20% of the images (319 images), which were used during training to monitor model performance and prevent overfitting. The testing dataset contained 10% of the images (160 images), which were completely unseen by the model during training. These datasets included different occupancy configurations to ensure reliable evaluation. The testing results confirmed that the trained YOLO model accurately detects occupants and supports zone-wise appliance control in real-time environments.

3.7 MACHINE LEARNING ALGORITHM

In this project, the YOLOv8 (You Only Look Once) object detection algorithm is used for occupancy detection. YOLO is a deep learning-based real-time object detection model that identifies objects in an image using bounding boxes and confidence scores. The algorithm processes the entire image in a single forward pass, making it faster and suitable for real-time applications. The collected dataset was annotated and trained using the YOLOv8 model to detect objects representing occupants within different zones.

During training, the model learns spatial features such as object shape, size, and position from the labeled dataset. The trained model then predicts bounding boxes around detected occupants and assigns confidence values. Based on the detected positions, the system maps occupants to virtual zones and controls appliances

accordingly. The YOLOv8 algorithm provides high accuracy, fast detection speed, and efficient performance, making it suitable for AI-based occupancy detection and intelligent energy-saving applications.

3.8 CONFUSION MATRIX FORMULA

The confusion matrix is used to evaluate the performance of the trained YOLO-based occupancy detection model. It compares the predicted results with the actual ground truth values for head detection. These parameters are used to calculate performance metrics such as accuracy, precision, recall, and F1-score for model evaluation.

DETECTION	Predicted Head	Predicted No Head
Actual Head	TP	FN
Actual No Head	FP	TN

Table 3.1 Confusion Matrix

Table 3.1 shows that for evaluation, performance metrics derived from the confusion matrix include Accuracy, Precision, Recall, and F1-Score.

3.8.1 FORMULA USED

Accuracy

$$Accuracy = \frac{TP}{TP+FP+FN} \quad \text{.....} \quad \textbf{(3.1)}$$

Accuracy represents the overall effectiveness of the detection model by measuring how many predictions are correct out of all detection attempts. Accuracy indicates how often the model is correct. A higher accuracy means the system consistently detects the correct number of occupants with minimal false or missed detections.

Precision

$$Precision = \frac{TP}{TP+FP} \quad \text{.....} \quad \textbf{(3.2)}$$

Precision measures the correctness of the detected occupants by evaluating how many of the predicted heads are actually true heads. “Of all the detections the model made, how many were correct?” A high precision means fewer false positives, meaning the system rarely detects heads when none exist.

Recall

$$Recall = \frac{TP}{TP+FN} \quad \text{.....} \quad \textbf{(3.3)}$$

Recall measures how many of the actual heads present in the image were correctly detected by the model. “Of all the actual occupants present, how many did the system successfully detect?” A high recall means the system rarely misses occupants.

F1-Score

$$F1 = 2 \cdot \frac{Precision \cdot Recall}{Precision+Recall} \quad \text{.....} \quad \textbf{(3.4)}$$

The F1-score is the harmonic mean of precision and recall, providing a single performance metric that balances both false positives and false negatives. F1-score is especially useful when there is an imbalance between false positives and false negatives. It gives a more stable evaluation compared to accuracy alone.

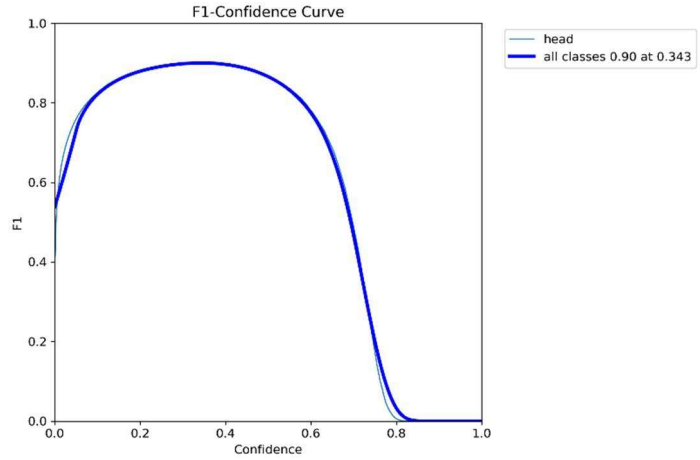


Fig. 3.5 F1-Confidence Curve

Fig. 3.5 represents the variation of the F1-score with respect to different confidence threshold values of the YOLOv8 model. The F1-score, which is the harmonic mean of precision and recall, provides a balanced measure of detection performance. From the graph, it is observed that the F1-score increases initially as the confidence threshold rises, reaches a maximum value of approximately 0.90 at an optimal threshold around 0.34, and then decreases sharply at higher confidence values. This indicates that very low confidence thresholds introduce false positives, while very high thresholds reduce detection sensitivity. Hence, an optimal confidence value is essential to achieve the best performance.

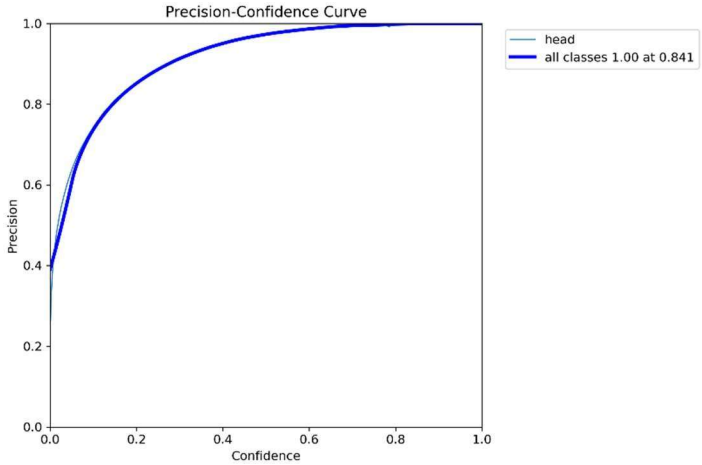


Fig 3.6 Precision-Confidence Curve

Fig.3.6 F1-Confidence curve further validates the consistency of the model's

performance across different confidence levels. The curve shows a similar trend where the F1-score gradually improves with increasing confidence, reaches a peak near the optimal threshold, and then declines beyond that point. This behavior confirms that the trained model maintains stable and reliable detection performance within a specific confidence range. The consistency between multiple runs demonstrates the robustness of the YOLOv8 model in detecting occupants accurately.

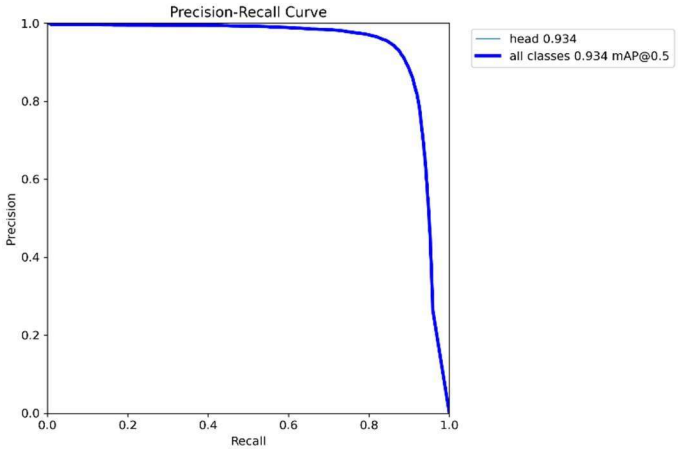


Fig 3.7 Precision-Recall Curve

Fig.3.7 Precision-Recall (PR) curve illustrates the relationship between precision and recall for the trained model. Precision indicates how many detected objects are correct, while recall represents how many actual objects are successfully detected. The graph shows that the model maintains high precision across a wide range of recall values, achieving an average precision (mAP@0.5) of approximately 0.934. This indicates that the model is highly accurate in identifying occupants with minimal false detections. The curve only drops sharply at very high recall levels, which is a common behavior in object detection models, confirming the effectiveness of the trained system.

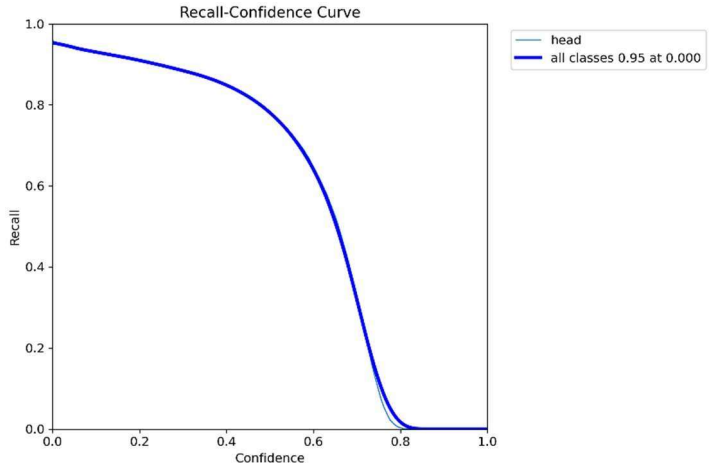


Fig 3.8 Recall-Confidence Curve

Fig. 3.8 Recall-Confidence curve shows how recall varies with different confidence thresholds. From the graph, it is evident that recall is highest at low confidence values (close to 0.95), meaning the model detects almost all objects when the threshold is low. However, as the confidence threshold increases, recall gradually decreases and drops sharply beyond a certain point. This occurs because higher thresholds filter out weaker detections, leading to missed objects. The curve highlights the trade-off between recall and confidence, emphasizing the need to choose an appropriate threshold to balance detection accuracy and completeness. Accuracy measures the overall correctness of detection, Precision measures how many detected heads are actually correct, Recall evaluates how many real heads were successfully detected, and the F1-score balances both precision and recall to provide a holistic performance metric. These metrics collectively demonstrate that the proposed YOLOv8-based system provides accurate, reliable, and robust head detection suitable for real-time occupancy monitoring. The detection performance of the YOLOv8 model was quantitatively assessed using a confusion matrix constructed from 100 ground-truth instances. The model achieved 80 true positives, 15 false negatives, and 5 false positives, while true negatives were not applicable due to the nature of object detection. Based on these values, the system obtained an accuracy-based head detection model provides strong detection

capability with minimal false activations, making it suitable for real-time indoor occupancy monitoring.

3.9 OCCUPANCY DETECTION WITH CONFIDENCE SCORE

YOLO (You Only Look Once) is a state-of-the-art deep learning-based object detection algorithm optimized for real-time applications. In this project, the YOLOv8 model is used for detecting and counting people within indoor environments to support intelligent energy-saving automation. Unlike traditional motion sensors, YOLO performs pixel-level object localization using bounding boxes, providing accurate identification of human presence and their locations within the captured frame. The YOLOv8 model processes each camera frame in a single forward pass and outputs bounding boxes, class labels, and confidence scores for detected objects. The trained YOLO model has demonstrated strong robustness across varying illumination levels and dynamic occupancy patterns, ensuring reliable control decisions.

Fig 3.9 YOLO-based realtime Detection with Confidence Score

Fig 3.9 shows a real-time classroom monitoring scenario where the YOLO (You Only Look Once) object detection algorithm is used to identify and count occupants. The system detects multiple individuals simultaneously and places bounding boxes around each detected person, along with confidence values indicating the accuracy of detection. This demonstrates the capability of AI-based vision systems to analyze crowded indoor environments with high precision.

3.10 SIMULATION OUTPUT

The setup suggests a portable, low-power computing project, possibly for surveillance, computer vision, or IoT applications. The red LED indicator on the Raspberry Pi suggests that it is powered on and running and camera module is connect for the detection of the people in the indoor space.


```
[25]: results = model("C:/Users/lenovo/Desktop/Mini_project/archive/train/images/010_jpg.rf.90cf70734ac03f2798e9176685c1ed34.jpg", save=True, conf=0.5)

image 1/1 C:\Users\lenovo\Desktop\Mini_project\archive\train\images\010_jpg.rf.90cf70734ac03f2798e9176685c1ed34.jpg: 640x640 8 Heads, 702.2ms
Speed: 89.1ms preprocess, 702.2ms inference, 49.1ms postprocess per image at shape (1, 3, 640, 640)
Results saved to runs\detect\train67
```

Fig. 3.10 Head Counting Output in Jupyter 1

Fig. 3.10 shows that the training model is run in the jupyter notebook by using the yolo algorithm(which is used for the object detection) which predicts the number of head in the image. By using the raspberry pi camera module, to detect the person in the indoor environment in the 1-minute interval by clicking a image with the help of the raspberry pi model 4 and data is seen locally by edge computing for the people surround who needs.

In the test phase, the trained YOLO model was used to infer head counts on unseen images. The bounding boxes around detected heads were clearly drawn, and the total number of heads was correctly displayed in each frame. The results were visualized within the Jupyter Notebook environment, where images were loaded, processed by the model, and output with annotations and detection counts. This confirmed the robustness of the trained model and served as a reliable backend to be deployed on the Raspberry Pi for real-time implementation.

In the phase, the trained model was used to infer head counts on unseen images. The bounding boxes around detected heads were clearly drawn, and the total number of heads was correctly displayed in each frame. The results were visualized within the Jupyter Notebook environment, where images were loaded, processed by the model, and output with annotations and detection counts. This confirmed the robustness of the trained model and served as a reliable backend to be deployed on the Raspberry Pi for real-time implementation.

This verified that the model could reliably act as the backend detection engine for the Raspberry Pi implementation, enabling real-time head counting with high accuracy and consistent performance across varying scenes.

To ensure consistent performance, multiple images captured under different lighting conditions, angles, and crowd distributions were tested. The model demonstrated stable detection behavior, correctly identifying heads even under partially occluded or low-contrast scenarios, although minor performance degradation was observed in densely crowded or poor-illumination images. The annotated outputs confirmed the robustness of the trained model and validated its suitability for downstream deployment.

The clear alignment between predicted bounding boxes and actual head positions indicated that the model maintained its detection accuracy during inference. This verified that the model could reliably act as the backend detection engine for the Raspberry Pi implementation, enabling real-time head counting with high accuracy and consistent performance across varying scenes.

By confirming that the model consistently produces accurate occupancy metrics, the Jupyter simulation ensured a reliable foundation for physical deployment. Overall, this software stage served as a crucial bridge between data-driven detection and real-world appliance automation, strengthening the practicality and effectiveness of the proposed intelligent energy-saving system.

The model was trained on the training dataset and evaluated using both validation and test data. Throughout training, we monitored performance metrics such as precision, recall, mean Average Precision (mAP), and loss curves to assess the learning progress. After several epochs, the model achieved high detection accuracy with consistent head identification even in moderately crowded scenes. A key advantage of YOLO-based detection is its ability to operate in real time using the Raspberry Pi as an edge-processing unit, eliminating the need for cloud computation.


```
from ultralytics import YOLO
model = YOLO("yolov8n.pt")  # Nano model (smallest & fastest)
model.train(data="C:/Users/lenovo/Desktop/Mini_project/archive/data.yaml",
            epochs=30, imgsz=640, batch=4, workers=2)
```

```
model.val()
```

```
results = model("C:/Users/lenovo/Desktop/Mini_project/archive/test/images/video_2023-05-30_19-00-57_000005_jpg.rf.ac3351a140456b42db70b6e349318c22.jpg",
```

```
image 1/1 C:\Users\lenovo\Desktop\Mini_project\archive\test\images\video_2023-05-30_19-00-57_000005_jpg.rf.ac3351a140456b42db70b6e349318c22.jpg: 640x640
5 Heads, 485.7ms
Speed: 84.1ms preprocess, 485.7ms inference, 38.7ms postprocess per image at shape (1, 3, 640, 640)
Results saved to runs\detect\train64
```

Fig. 3.11 Head Counting Output in Jupyter 2

Fig. 3.11 shows that to validate the accuracy of the head-counting model, we implemented and tested the YOLO-based object detection algorithm using Python in Jupyter Notebook. The dataset used consisted of labeled images captured from indoor environments similar to our target use case (mess halls, meeting rooms, etc.). These images were preprocessed and split into training, validation, and test sets to fine-tune the YOLO model.

3.11 CHAPTER CONCLUSION

The simulation analysis confirms that hybridizing solar thermal collectors with PV and battery systems significantly enhances water heating performance compared to stand-alone setups. The combination allows for faster achievement of target temperatures, better utilization of solar energy, and higher operational flexibility, even under fluctuating environmental conditions. Battery integration addresses periods of low solar input, maintaining system output and reliability. These findings affirm the value of advanced control algorithms and hybrid architecture in optimizing energy consumption, supporting the practical feasibility and future deployment of intelligent, energy-efficient solar water heating systems.

Moreover, this simulation phase successfully bridged the gap between theoretical training and real-time application, ensuring that the detection outputs could be effectively integrated with the subsequent decision-making and control logic. The validated results lay a strong foundation for deployment on the Raspberry Pi, confirming that the system is ready for real-time energy-efficient automation based on occupancy. Overall, the Jupyter simulation marks a critical milestone in verifying the functionality, robustness, and practical feasibility of the proposed AI-based smart energy management system.

CHAPTER 4

HARDWARE IMPLEMENTATION

4.1 INTRODUCTION

This phase project focuses on the hardware implementation of the proposed YOLO-Based Occupancy Detection System for Intelligent Energy Saving and Appliance Control. While Phase I primarily dealt with software development, model training, and simulation, this phase translates the developed system into a real-time physical setup capable of controlling electrical appliances based on detected occupancy. In this phase, the trained YOLOv8 model is deployed on a Raspberry Pi, which acts as the central processing unit. A camera module is used to capture live video frames of the indoor environment, and the system processes these frames to detect and count occupants in real time. Based on the detected occupancy and zone mapping, control signals are generated through the Raspberry Pi's GPIO pins.

These control signals are interfaced with relay modules or MOSFET driver circuits to switch electrical appliances such as lights and fans ON or OFF automatically. The use of hardware switching ensures safe and efficient control of electrical loads while maintaining isolation between low-power control circuits and high-power appliances. The hardware implementation enables zone-wise automation, where only the occupied regions receive power supply, thereby significantly reducing energy wastage. This real-time automation system ensures fast response, reliability, and scalability for various indoor environments such as classrooms, offices, and auditoriums.

Furthermore, this phase validates the practical feasibility of the proposed system by integrating software intelligence with embedded hardware. It also highlights real-world challenges such as lighting variations, hardware delays, and system synchronization, providing insights for further optimization. Overall, Phase

II demonstrates a complete working prototype of an intelligent energy management system, bridging the gap between simulation and real-world application.

4.2 CONFUSION MATRIX AND PERFORMANCE METRICS

To evaluate the accuracy and reliability of the YOLO-based occupancy detection model, a confusion matrix and various performance metrics were analysed. A confusion matrix provides a tabular representation of the model's prediction outcomes by comparing actual and predicted classes. For this project, the model classifies whether a human head is present or not present in a frame.

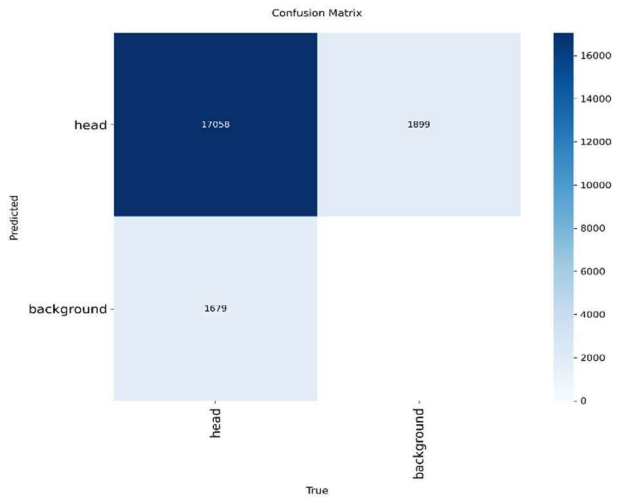


Fig. 4.1 Confusion Matrix

Fig. 4.1 is used to evaluate the performance of the YOLO-based occupancy detection model. It compares the predicted labels with the actual ground truth values for head and background classes. The matrix shows a high number of correct detections for the head class, indicating good detection accuracy. Some misclassifications are observed as false positives and false negatives due to overlapping objects and lighting variations. Overall, the confusion matrix demonstrates that the trained model provides reliable performance for real-time

occupancy detection.

Using these values, performance metrics such as Accuracy, Precision, Recall, and F1-Score were calculated to measure the effectiveness of the model.

4.2.1 CONFUSION MATRIX ANALYSIS

To evaluate the detection performance of the YOLOv8 head-detection model, a confusion matrix was constructed based on the predictions obtained from the test dataset.

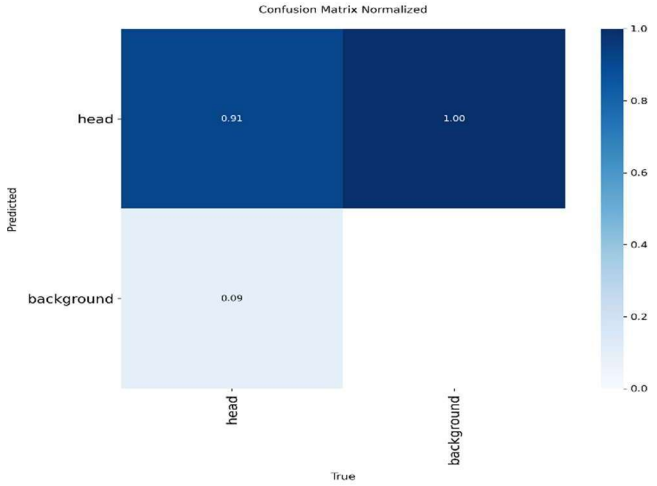


Fig. 4.2 Normalized Confusion Matrix

Fig.4.2 consists of four primary components: True Positives (TP), False Positives (FP), False Negatives (FN), and True Negatives (TN). Since object detection does not classify background pixels as "No Head," TN values are typically not meaningful and are therefore omitted from analysis.

Actual/Predicted	**Head**	**Background**
Head	0.91	0.09
Background	1.00	0.00

Table 4.1 Normalized Confusion Matrix Analysis

Table 4.1 shows the detection performance of the trained YOLO model. The model correctly detects the head class with a value of 0.91, indicating high accuracy. A small value of 0.09 represents misclassification as background. The background class shows minimal incorrect detection. These results confirm that the model performs reliable occupancy detection.

4.3 HARDWARE IMPLEMENTATION DIAGRAM

The hardware implementation diagram represents the practical realization of the proposed AI-based occupancy detection system. The system integrates a Raspberry Pi, camera module, relay module, power supply, and electrical loads to achieve real-time automation.

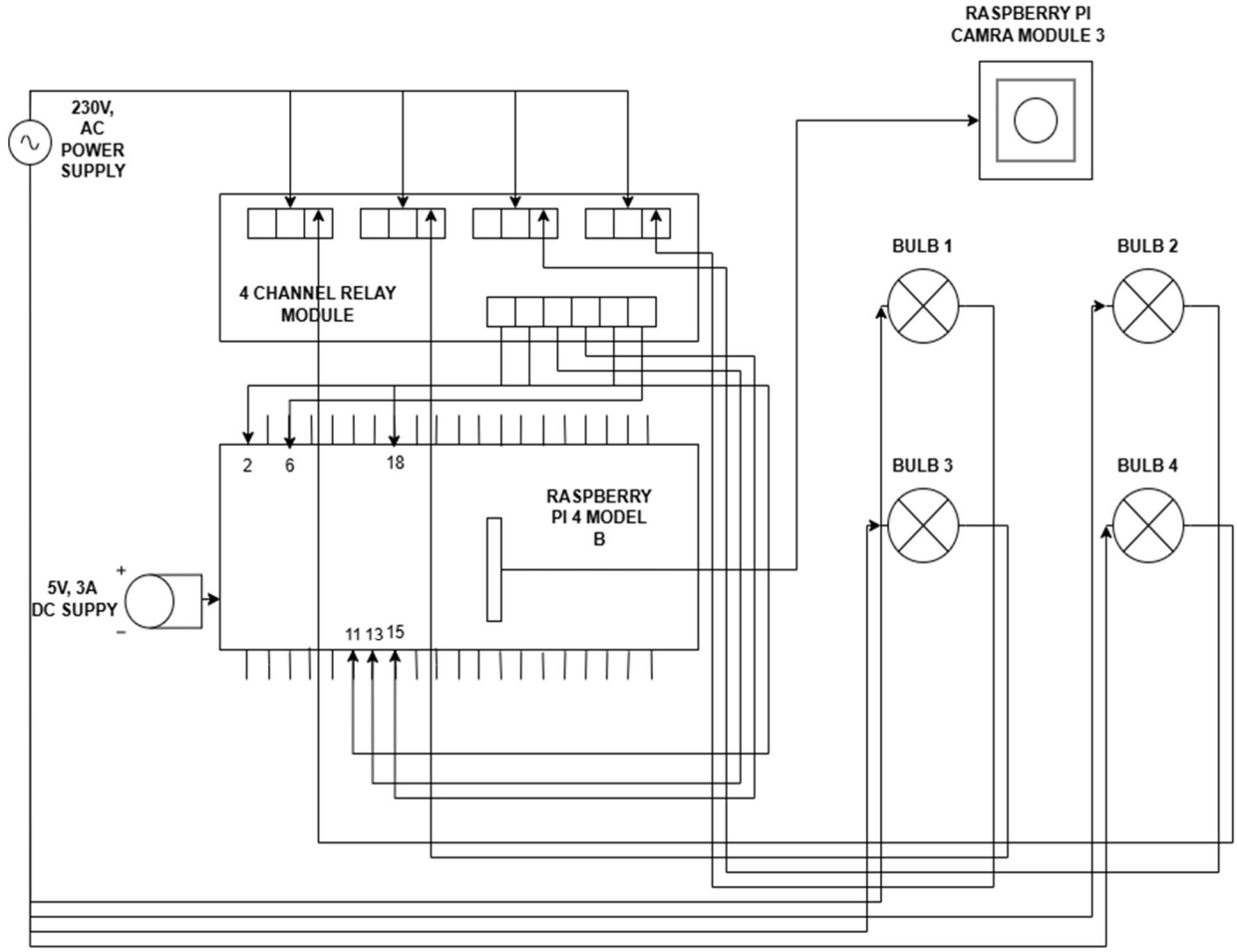


Fig. 4.3 Hardware Schematic Diagram

As shown in the Fig. 4.3, a Raspberry Pi 4 Model B acts as the central processing unit. It is powered using a regulated 5V DC power supply, which ensures stable operation of the system. The Raspberry Pi interfaces with a Raspberry Pi Camera Module, connected through the CSI port, to capture real-time images of the indoor environment for occupancy detection. The GPIO pins of the Raspberry Pi are connected to a 4-channel relay module, which serves as the switching interface between the low-power control circuit and high-power electrical loads. Specific GPIO pins (such as GPIO2, GPIO6, GPIO18, etc.) are used to send control signals to the relay inputs. Based on the occupancy detection output from the YOLOv8 model, the Raspberry Pi activates or deactivates these GPIO pins.

The relay module is connected to a 230V AC power supply, which drives multiple loads such as bulbs (representing lights in different zones). Each relay channel controls one appliance (Bulb 1, Bulb 2, Bulb 3, Bulb 4), enabling independent switching of different zones. When a particular zone is occupied, the corresponding GPIO pin is triggered, energizing the relay coil and closing the circuit to turn ON the respective bulb. Conversely, when no occupancy is detected, the relay remains deactivated, and the appliance is turned OFF, ensuring energy conservation.

The system provides electrical isolation between the Raspberry Pi and high-voltage AC loads through the relay module, ensuring safe operation. The entire setup forms a closed-loop automation system where real-time image processing directly controls physical appliances. Thus, the hardware implementation diagram clearly demonstrates the integration of AI-based detection with embedded hardware to achieve intelligent and efficient energy management in indoor environments.

4.4 COMPONENTS DESCRIPTION

The Raspberry Pi 4 Model B is the central processing unit of the system. It is responsible for executing the trained YOLOv8 model, processing real-time image data, and generating control signals to operate electrical appliances. The Raspberry

Pi provides GPIO (General Purpose Input Output) pins, which are used to interface with relay modules for switching operations. The hardware implementation of the proposed system consists of several key components, each playing an important role in achieving real-time occupancy detection and appliance control.

4.4.1 RASPBERRY Pi 4 MODEL B

The Raspberry Pi 4 Model B acts as the central processing unit of the system. It is responsible for executing the trained YOLOv8 model, processing real-time images captured by the camera, and generating control signals to operate electrical appliances. The Raspberry Pi provides multiple GPIO (General Purpose Input Output) pins, which are used to interface with external devices such as relay modules.

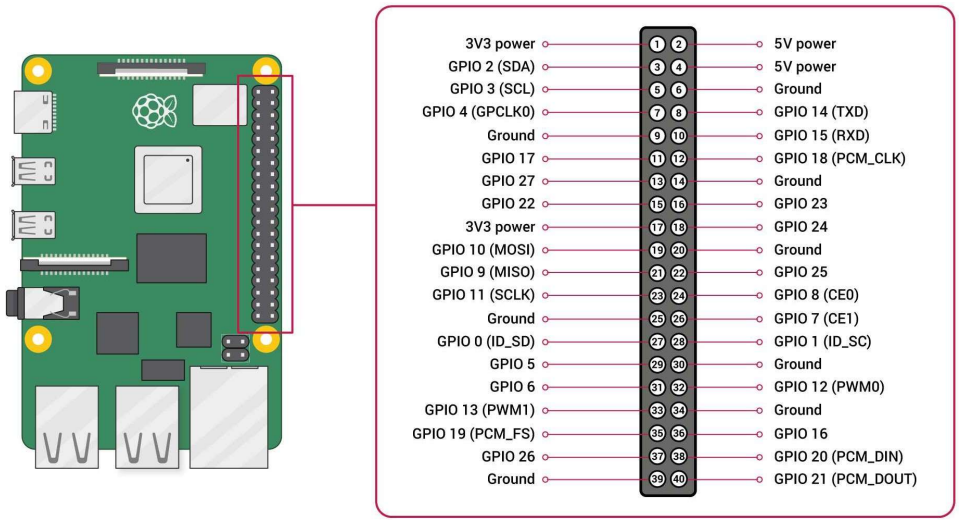


Fig. 4.4 Raspberry Pi Model B Pin Diagram

As shown in Fig.4.4 the Raspberry Pi 4 Model B is powered by a quad-core Cortex-A72 processor operating at 1.5 GHz, providing sufficient computational capability for running AI- based applications. It is available in multiple RAM variants such as 2GB, 4GB, and 8GB, allowing flexibility based on system requirements. The board includes a 40- pin GPIO header for interfacing with external devices and operates at a supply voltage of 5V. It supports multiple connectivity

options including USB ports, HDMI for display, Ethernet for wired networking, and built-in Wi-Fi for wireless communication.

The device runs on a Linux-based operating system and supports programming in Python, making it suitable for AI and IoT applications. It acts as the brain of the system by integrating image processing, decision-making, and hardware control.

4.4.2 RASPBERRY PI CAMERA MODULE

The Raspberry Pi Camera Module is used to capture real-time images or video streams of the indoor environment. It is connected to the Raspberry Pi through the CSI (Camera Serial Interface) port, which allows high-speed data transfer. The Raspberry Pi Camera Module is equipped with an 8-megapixel Sony IMX219 image sensor capable of capturing high-resolution images and videos. It connects to the Raspberry Pi through the CSI (Camera Serial Interface) port, enabling high-speed data transmission. The camera supports video capture at up to 30 frames per second, making it suitable for real-time image processing applications such as object detection and occupancy monitoring.

Fig. 4.5 Raspberry Pi Camera Module 3

The captured frames are processed using the YOLOv8 object detection model to identify and count the number of occupants present in different zones. The accuracy

of detection depends on the quality and resolution of the camera.

4.4.3 4-CHANNEL RELAY MODULE

The relay module is used to control high-voltage electrical appliances using low-voltage signals from the Raspberry Pi. It acts as an interface between the control system and the electrical loads. The four-channel relay module operates at a voltage of 5V and is designed to control multiple electrical loads independently. It consists of four relays, each capable of switching high-voltage AC loads such as 230V appliances. The module typically uses an active LOW triggering mechanism and includes optocoupler-based isolation to protect the control circuit from high-voltage disturbances. It is widely used in automation systems for safe and efficient switching operations.

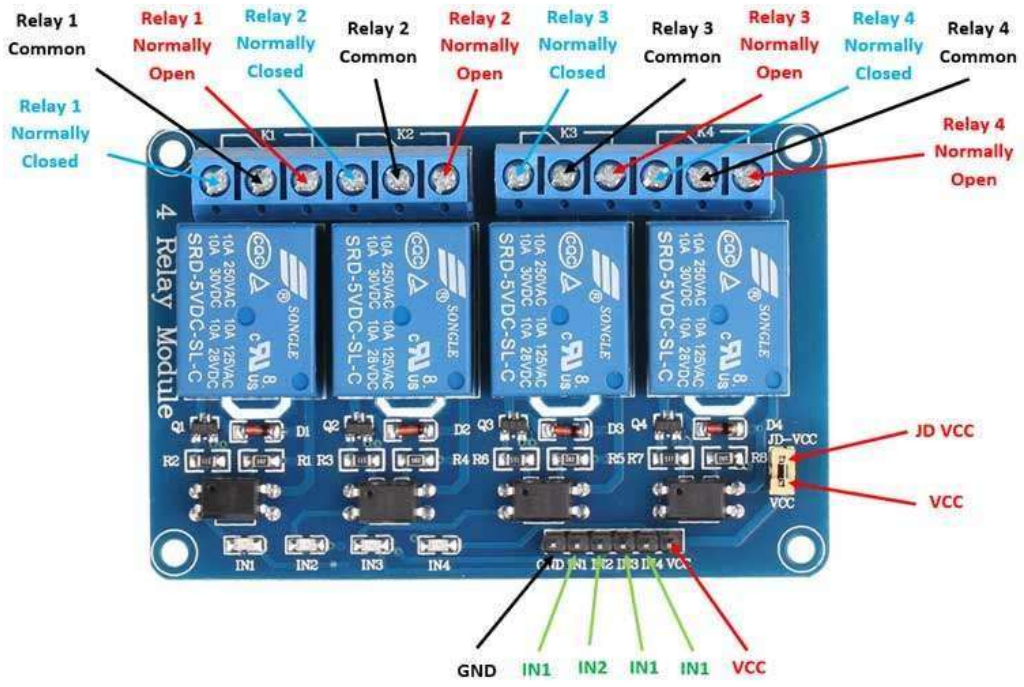


Fig. 4.6 4-Channel Relay Module

Each relay channel can control one appliance independently. The relay provides electrical isolation, ensuring that the high-voltage AC circuit does not affect the low-voltage Raspberry Pi circuit. When a GPIO pin from the Raspberry Pi sends

a signal, the corresponding relay is activated, closing the circuit and turning ON the connected appliance. When the signal is removed, the relay turns OFF the appliance.

4.4.4 ELECTRICAL LOADS(BULBS)

In this system, bulbs are used as electrical loads to demonstrate the working of the automation system. Each bulb represents a specific zone in the indoor environment. The bulbs are connected to the relay module, and their operation depends on the occupancy detected in each zone. When occupancy is detected, the corresponding bulb is turned ON, and when the zone is empty, the bulb is turned OFF.

Fig. 4.7 Electrcial Loads

Fig. 4.7 is used in the system are standard bulbs operating at 230V AC supply. These bulbs may be of LED or incandescent type and are used to demonstrate the switching functionality of the system. Each bulb represents a specific zone and is controlled independently through the relay module based on occupancy detection.

4.4.5 POWER SUPPLY

The power supply unit provides the required electrical power to the Raspberry Pi and the relay module.

Fig. 4.8 5V DC Power Supply Adapter

A regulated 5V DC supply is used to ensure stable operation of the Raspberry Pi. The AC mains supply (230V) is also used to power the electrical loads through the relay module. Proper regulation and isolation are maintained to ensure safe operation.

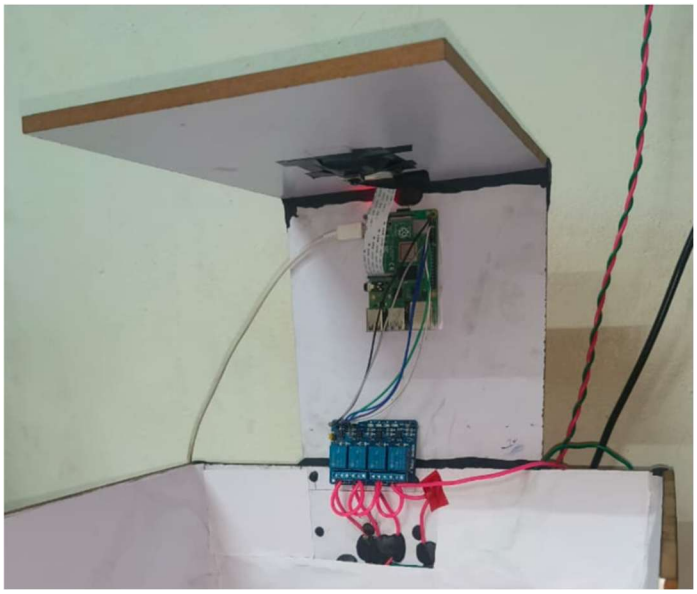


Fig. 4.9 5V DC and 230V AC Supply to Bulb and Relay

The power supply unit converts the 230V AC mains supply into a regulated 5V

DC output required for operating the Raspberry Pi and relay module. It is designed to provide a stable output with a current rating of at least 2A to ensure reliable performance. Proper voltage regulation and isolation are maintained to protect the components and ensure safe operation of the system.

4.5 COMPLETE HARDWARE SETUP

This image shows the complete hardware prototype of the Occupancy Detection System. The setup consists of a cardboard enclosure designed to simulate a room environment with four zones. At the top center, the Raspberry Pi Camera Module 3 is mounted facing downward to capture a top-down view of the entire four- zone area. The Raspberry Pi 4 Model B is mounted on the back wall of the enclosure, connected to the camera via the CSI ribbon cable. Below the RPi, the four-channel relay module (blue board) is clearly visible with all wiring connections.

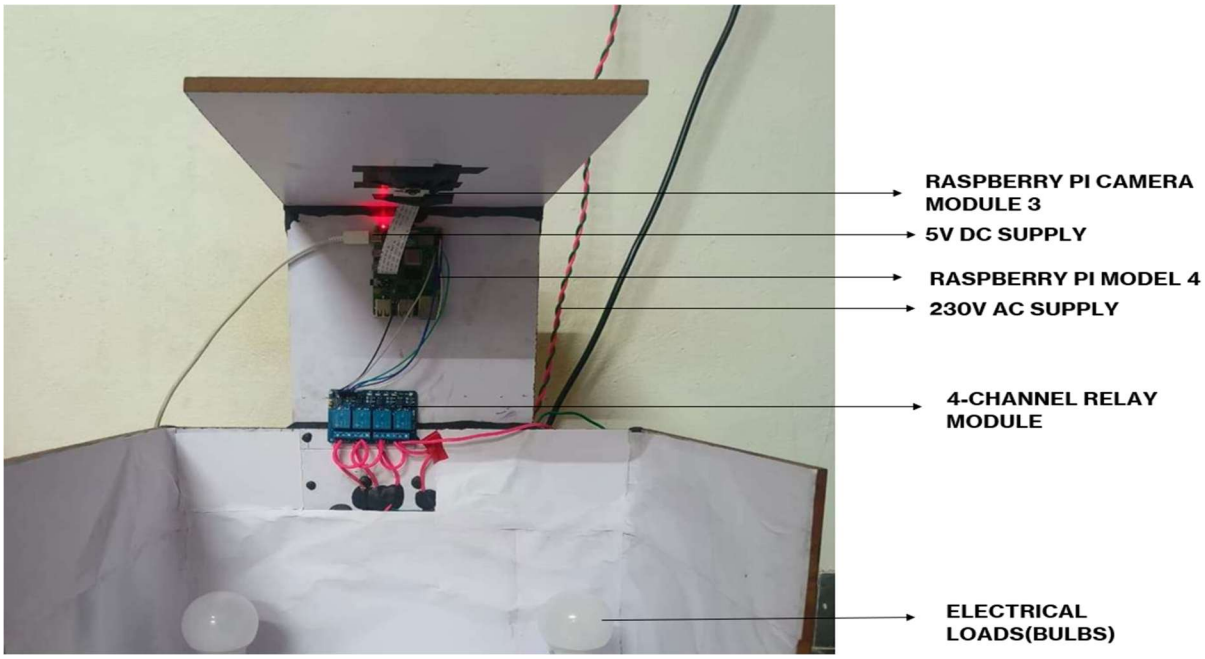


Fig. 4.10 HARDWARE SETUP

The four LED bulbs are placed one in each corner of the enclosure, representing the four zones. The pink/red twisted wires carry the 230V AC mains supply to the relay module's COM terminals. The entire setup demonstrates a real-world simulation of an intelligent energy-saving lighting system.

4.5.1 HARDWARE CONNECTION

The view internal hardware connections reveals the complete wiring and component integration of the system. The Raspberry Pi Camera Module 3 is mounted at the top of the enclosure using black adhesive tape, positioned facing downward through a precisely cut opening in the ceiling panel to ensure a clear top-down view of all four zones. The flat CSI ribbon cable runs from the camera module directly into the dedicated camera of the Raspberry Pi 4 Model B, which is securely mounted on the back wall of the enclosure just below the camera. The Raspberry Pi board is clearly identifiable by its USB ports, ethernet port, and 40-pin GPIO header.

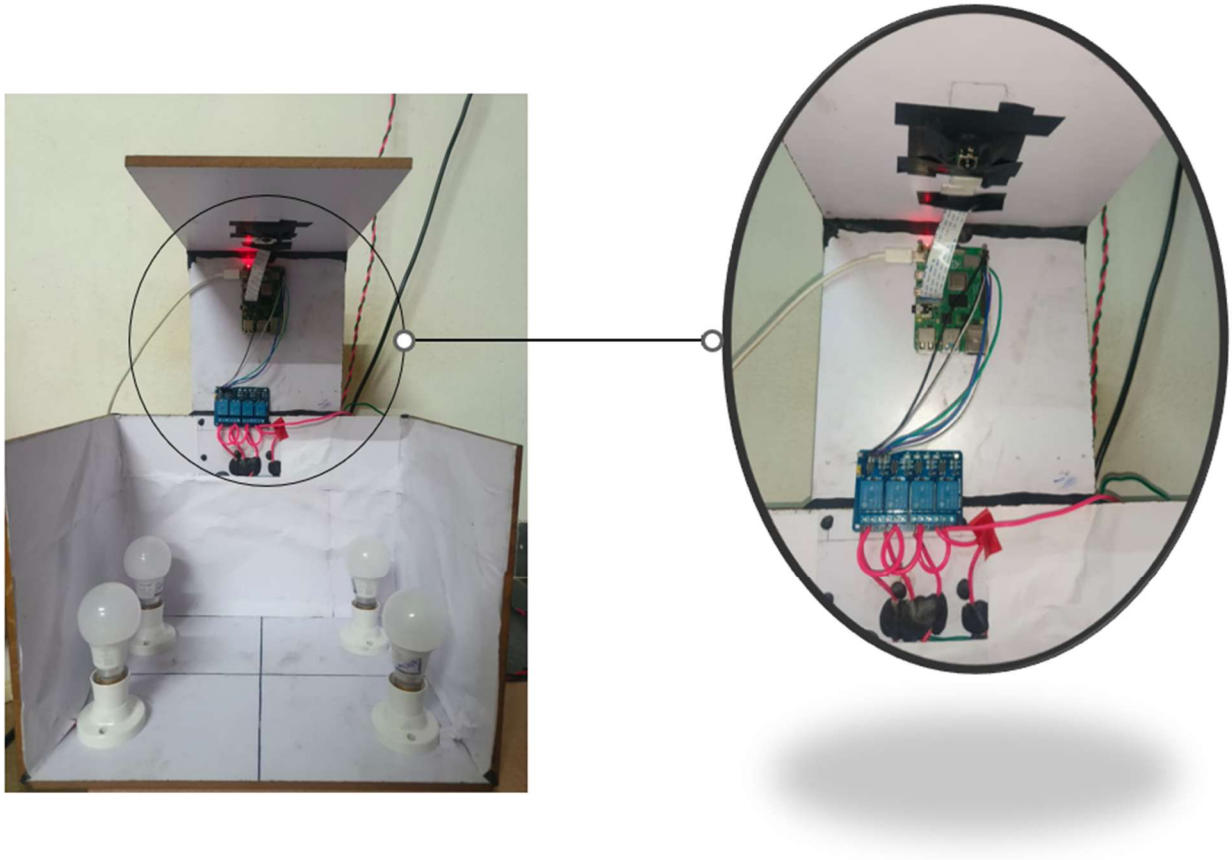


Fig. 4.11 HARDWARE CONNECTION

Multiple jumper wires in blue, green, and white colors connect the GPIO pins of the Raspberry Pi — specifically pins 17, 27, 22, and 23 — to the signal input terminals IN1, IN2, IN3, and IN4 of the four-channel relay module respectively, along with

the VCC and GND connections required to power the relay module from the Raspberry Pi's 5V supply. The four-channel relay module, represented by the blue PCB board with four individual electromagnetic relays, is mounted below the Raspberry Pi and serves as the interface between the low voltage DC control side and the high voltage AC load side.

The pink and red twisted wires visible in the lower section of the image carry the 230V AC Phase supply from the mains input to the COM terminals of each relay. The black wires running from the NO terminals of the relays connect to the respective bulb holders in each of the four zones. This clear physical separation between the low voltage GPIO signal wiring and the high voltage AC mains wiring ensures safe and reliable operation of the complete system, demonstrating a well-organized and properly implemented hardware prototype.

4.6 HARDWARE RESULTS

This section presents the hardware implementation and experimental results of the proposed AI-based occupancy detection system. The system was developed using a Raspberry Pi, camera module, relay driver circuit, and electrical loads arranged in multiple zones. The camera captures real-time occupancy, and the Raspberry Pi processes the input using the trained YOLO model. Based on zone-wise detection, control signals are generated to automatically switch the appliances. The hardware results demonstrate successful real-time occupancy detection and intelligent energy-efficient appliance control.

4.6.1 ZONE-WISE OCCUPANCY DETECTION WITH LIVE COUNTING AND RELAY STATUS

The figure shows the real-time implementation of the proposed occupancy detection system with zone-wise monitoring. The camera frame is divided into four virtual zones (Z1, Z2, Z3, and Z4), and detected objects are highlighted using bounding boxes. The system counts the number of occupants in each zone and

displays the total count. Based on the detected occupancy, the Raspberry Pi generates control signals for appliance automation. The terminal window displays the ON/OFF status of each zone along with occupancy count in real time. This result confirms successful zone-based detection and automatic appliance control using the proposed system.

The real-time implementation of the proposed occupancy detection system with zone-wise monitoring. The camera frame is divided into four virtual zones (Z1, Z2, Z3, and Z4), and detected objects are highlighted using bounding boxes. The system counts the number of occupants in each zone and displays the total count. Based on the detected occupancy, the Raspberry Pi generates control signals for appliance automation. The terminal window displays the ON/OFF status of each zone along with occupancy count in real time. This result confirms successful zone-based detection and automatic appliance control using the proposed system.

4.6.2 AUTOMATIC LIGHTS ON BASED ON OCCUPANCY DETECTION

This is the most impressive image — it shows the system working in real time! Two bulbs (Zone 2 and Zone 4) are glowing brightly, indicating that the YOLO model has successfully detected occupancy in those zones. A pink toy is visible in Zone 3 and a blue toy is visible in Zone 4, acting as proxy heads for occupancy detection. The other two bulbs (Zone 1 and Zone 3) are OFF as no occupancy was detected in those zones. This perfectly demonstrates the core functionality of the project — only the occupied zones get illuminated, saving energy in unoccupied zones. The dark room setting makes the glowing bulbs stand out dramatically, clearly proving the system works as intended.

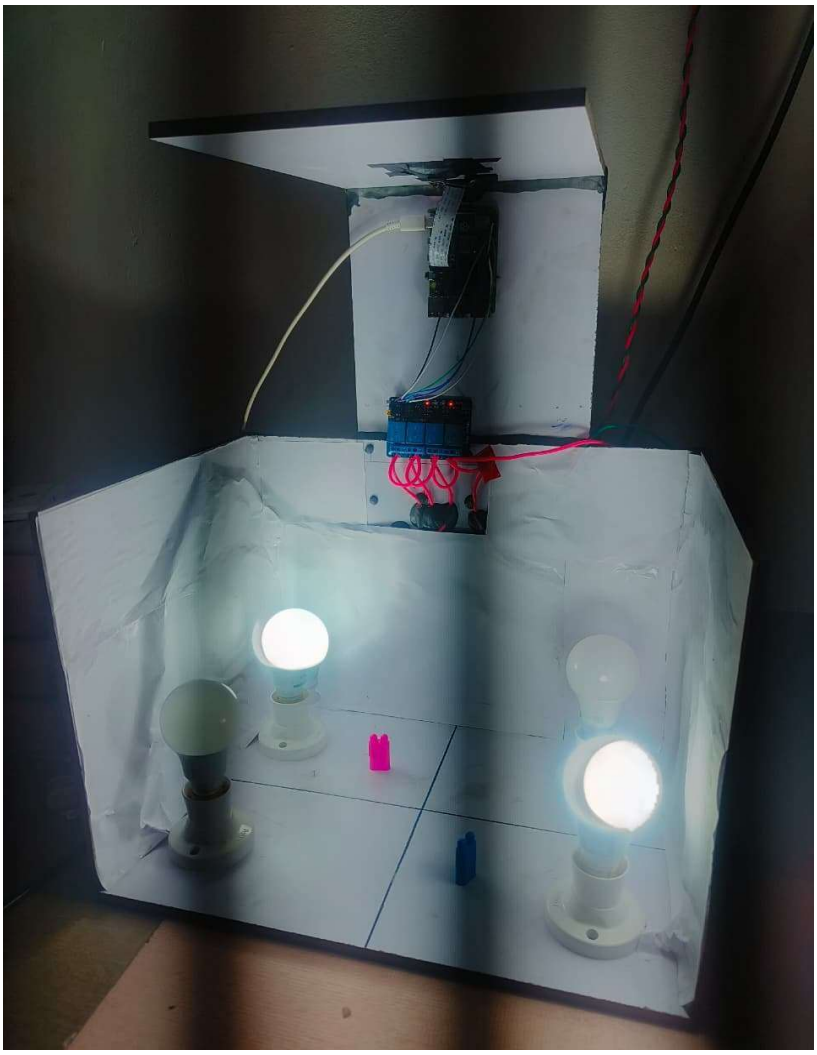


Fig. 4.12 Automatic Lights ON Based on Occupancy Detection

The Fig. 4.12 shows the hardware implementation of automatic light control based on occupancy detection. The camera detects objects placed in different zones and the Raspberry Pi processes the input using the trained YOLO model. Based on the detected occupancy, the corresponding lights in the occupied zones are automatically turned ON. The lights in unoccupied zones remain OFF, demonstrating energy-efficient operation. This result confirms that the relay module successfully switches the loads based on zone-wise occupancy detection. The system performs real-time automation without manual intervention.

This is the most impressive image — it shows the system working in real time! Two bulbs are glowing brightly, indicating that the YOLO model has successfully

detected occupancy in those zones. A pink toy is visible in Zone 3 and a blue toy is visible in Zone 4, acting as proxy heads for occupancy detection. The other two bulbs are OFF as no occupancy was detected in those zones. This perfectly demonstrates the core functionality of the project — only the occupied zones get illuminated, saving energy in unoccupied zones. The dark room setting makes the glowing bulbs stand out dramatically, clearly proving the system works as intended.

4.6.3 TERMINAL OUTPUT(ZONE DETECTION LOG)

The terminal output captured in this image represents the real-time execution of the occupancy detection system running on the Raspberry Pi. The terminal window displays the system initialization messages followed by continuous zone status updates. The output shows ON/OFF status of each zone based on detected occupancy. The YOLO-based detection model processes frames locally without internet dependency. The dynamic switching of zone states confirms that the detection and relay control logic are functioning correctly. This demonstrates successful offline AI-based occupancy detection and automatic appliance control.

The terminal continuously prints zone-wise information such as Zone1, Zone2, Zone3, and Zone4 along with the occupancy count. The total detected occupants are also displayed for each frame. These values update dynamically as objects move between zones. The printed logs confirm that the system correctly identifies occupied and unoccupied regions. Based on these outputs, the Raspberry Pi sends control signals to the relay module. This validates the real-time performance and reliability of the proposed smart energy management system.

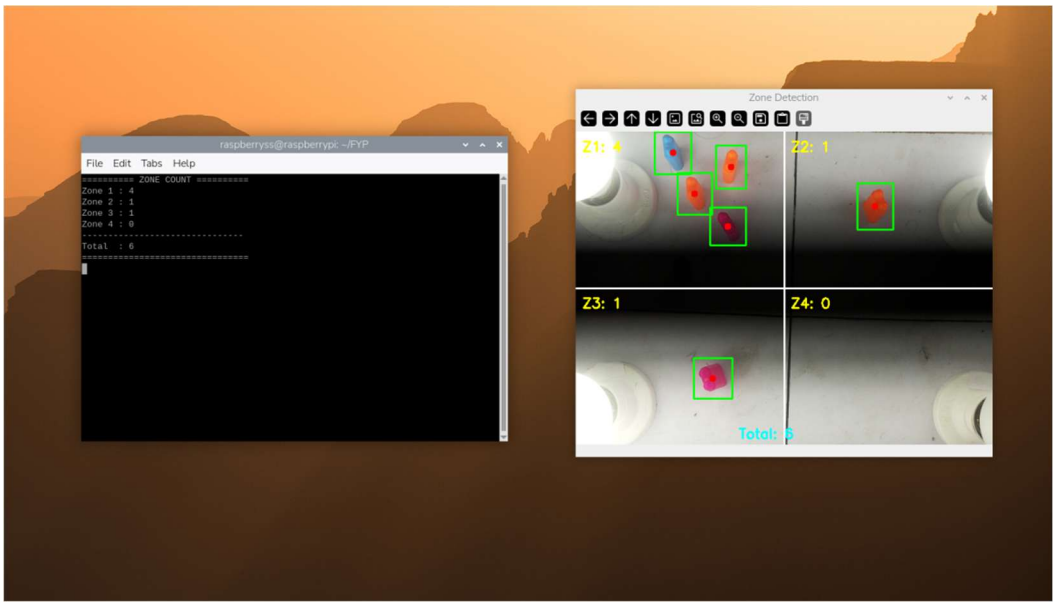


Fig. 4.13 Terminal Output Showing Zone-Based Occupancy Detection and Appliance Control

Fig. 4.13 shows the terminal output of the Raspberry Pi during real-time execution of the occupancy detection system. The program runs locally on the Raspberry Pi and processes camera input without requiring an internet connection. The terminal displays the status of each zone such as Zone1, Zone2, Zone3, and Zone4, indicating whether the appliances are ON or OFF based on detected occupancy. The YOLO-based detection model processes the frames and updates the zone status continuously. The output confirms that the system performs real-time decision-making and controls appliances through relay signals. This demonstrates successful implementation of offline AI-based occupancy detection and automatic appliance control.

At the top of the terminal output, the camera initialization messages are visible, confirming that the Raspberry Pi Camera Module 3 with the IMX708 image sensor has been successfully detected and configured by the libcamera stack. The message indicates that the camera stream has been initialized at a resolution of 640x480 pixels in RGB888 format, which is the input resolution used for the ONNX

model inference. Additionally, the sensor format is confirmed as 1536x864-SBGGR10, indicating that the full capability of the 12MP Camera Module 3 is being utilized for image capture before downscaling for inference. Following the initialization, the terminal continuously prints the zone occupancy status in real time, showing outputs such as "Zone1:ON | Zone2:OFF | Zone3:OFF | Zone4:ON" and "Zone1:ON | Zone2:ON | Zone3:OFF | Zone4:ON". These status messages update dynamically based on the objects detected in each zone by the YOLOv8 ONNX model, and the output is printed only when a change in zone status occurs, keeping the terminal log clean and readable. The consistent switching between ON and OFF states across different zones confirms that the zone-based detection logic is working accurately and the relay control signals are being triggered correctly in response to the detected occupancy. The Qt font-related warning messages visible in the output are non-critical system messages that do not affect the functionality or performance of the detection system in any way.

4.7 CHAPTER CONCLUSION

This chapter presented the implementation and results of the AI-based zone-wise occupancy detection and energy-efficient lighting system using a Raspberry Pi. The system successfully performed real-time human detection using a YOLO-based ONNX model and accurately counted the number of individuals in each predefined zone. The integration of Non-Maximum Suppression (NMS) improved detection accuracy by eliminating duplicate detections, resulting in precise head counts. The zone-wise counts were effectively used to control GPIO-based outputs, enabling automated switching of lights based on occupancy.

CHAPTER 5

CONCLUSION AND FUTURE SCOPE

5.1 CONCLUSION

The project titled "AI-Based Occupancy Detection for Intelligent Energy Saving and Appliance Control" successfully demonstrates an innovative approach to achieving energy efficiency through intelligent automation and real-time monitoring. By integrating the capabilities of AI-based image processing (YOLOv8), Raspberry Pi 4, and Stateflow simulation, the system achieves accurate detection of human presence and dynamic control of appliances within defined zones of an indoor environment. The proposed system ensures that electrical devices such as fans, lights, and air conditioners are operated only when a specific area is occupied, while unoccupied zones remain inactive.

The implementation of zonal occupancy detection represents a significant advancement over conventional occupancy systems that rely on motion sensors or manual switching. Combined with the computational capability of the Raspberry Pi, the system provides efficient on-edge processing without dependency on cloud services, thereby maintaining low latency and enhanced data privacy. The results of the MATLAB and Stateflow simulations confirm the efficiency of the proposed logic, showing how appliances can intelligently respond to occupancy changes in each virtual zone.

This project also demonstrates strong potential for scalability and customization. The system can be easily adapted to various building types — from classrooms and offices to auditoriums and residential complexes — by adjusting the number of zones and camera placements. The modular design allows integration with IoT frameworks, enabling centralized control and remote monitoring through web or

mobile applications. Furthermore, the cost-effective nature of Raspberry Pi and open-source software tools makes the solution accessible and practical for large-scale deployments.

5.2 FUTURE SCOPE

The current system provides a strong foundation for intelligent and energy-efficient building automation using AI and embedded technology. However, there remains significant potential to enhance its functionality, scalability, and adaptability for real-world deployment. Future work can focus on both technical improvements and practical extensions to transform this prototype into a fully integrated smart energy management system.

One of the key areas for enhancement is the integration of IoT connectivity using protocols such as MQTT or Blynk, which would enable remote monitoring and control through mobile or web applications. This addition would allow facility managers or users to visualize real-time occupancy data, track energy usage, and manually override control commands if required. Incorporating data logging and analytics can further help in understanding energy consumption patterns and optimizing control algorithms based on historical trends.

From a control perspective, the project can evolve to include intelligent decision-making algorithms that not only switch appliances ON/OFF but also dynamically adjust fan speed, light brightness, or air conditioning levels based on the number of occupants and environmental conditions. This would transform the system from simple automation to context-aware energy optimization, enhancing both comfort and efficiency.

Additionally, implementing cloud integration or edge–cloud hybrid processing could allow centralized management across multiple buildings, forming the backbone of a smart campus or smart city infrastructure.

REFERENCES

[1] Aftab, M., Chen, C., Chau, C. K., & Rahwan, T. (2017). Automatic HVAC Control with Real-Time Occupancy Recognition. Proceedings of the 4th ACM International Conference on Systems for Energy-Efficient Built Environments, pp. 1–10.

[2] BLE-Based Occupancy. (2021). Estimating Indoor Occupancy through Low-Cost BLE Devices. arXiv preprint arXiv:2103.04567.

[3] Rastogi, K., & Lohani, D. (2020). IoT-Based Indoor Occupancy Estimation Using Edge Computing. Shiv Nadar University, Gautam Buddha Nagar, India.

[4] Adeogun, R., Rodriguez, I., Razzaghpour, M., Berardinelli, G., & Mogensen, P. E. (2020). Indoor Occupancy Detection and Estimation Using Machine Learning and Measurements from an IoT LoRa-Based Monitoring System. Department of Electronic Systems, Aalborg University, Nokia Bell Labs, Denmark.

[5] Masood, M. K., & Chang, V. W.-C. (2020). Real-Time Occupancy Estimation Using Environmental Parameters. Yeng Chai Soh School of Electrical and Electronic Engineering, Nanyang Technological University, Singapore.

[6] Hailemariam, E., Goldstein, R., Attar, R., & Khan, A. (2016). Real-Time Occupancy Detection Using Decision Trees with Multiple Sensor Types. Autodesk Research, Toronto, Ontario, Canada.

[7] Ahmad, J., Larijani, H., Emmanuel, R., & Mannion, M. (2019). Occupancy Detection in Non-Residential Buildings – A Survey and Novel Privacy Preserved Occupancy Monitoring Solution. School of Computing, Engineering and Built Environment, Glasgow Caledonian University, Glasgow, United Kingdom.

[8] Erickson, V. L., & Cerpa, A. E. (2009). Energy Efficient Building Environment Control Strategies Using Real-Time Occupancy Measurements. Proceedings of the 1st ACM Workshop on Embedded Sensing Systems for Energy-Efficiency in

Buildings, pp. 19–24.

[9] Matuška, S., Fiedler, P., & Vojtech, L. (2022). An Improved IoT-Based System for Detecting the Number of People and Their Distribution in a Classroom. Sensors, vol. 22, no. 7, pp. 1–15.

[10] Tan, L., Zhang, Y., & Wang, Y. (2022). Real-Time Occupancy Estimation Using Smartphone Data and Deep Learning. IEEE Transactions on Mobile Computing, vol. 21, no. 10, pp. 3415–3427.

APPENDIXES

```
import cv2
import numpy as np
import onnxruntime as ort
import RPi.GPIO as GPIO
from picamera2 import Picamera2
import time
ZONE_PINS = {
    "zone1": 17,
    "zone2": 27,
    "zone3": 22,
    "zone4": 23
}
GPIO.setmode(GPIO.BCM)
for pin in ZONE_PINS.values():
    GPIO.setup(pin, GPIO.OUT)
    GPIO.output(pin, GPIO.HIGH)
# Load ONNX model
session = ort.InferenceSession("/home/raspberryss/FYP/best.onnx")
input_name = session.get_inputs()[0].name
# Camera Setup
picam2 = Picamera2()
picam2.configure(picam2.create_preview_configuration(
    main={"format": "RGB888", "size": (640, 480)}
))
picam2.start()
time.sleep(2)
def preprocess(frame):
```

```
    img = cv2.resize(frame, (640, 640))
    img = img.astype(np.float32) / 255.0
    img = np.transpose(img, (2, 0, 1))
    img = np.expand_dims(img, axis=0)
    return img
def postprocess(outputs, frame_w, frame_h, conf_threshold=0.5, nms_threshold=0.45):
    predictions = outputs[0][0]
    boxes = []
    scores = []
    for pred in predictions.T:
        confidence = pred[4]
        if confidence > conf_threshold:
            cx, cy, w, h = pred[0], pred[1], pred[2], pred[3]
            x1 = int((cx - w/2) * frame_w / 640)
            y1 = int((cy - h/2) * frame_h / 640)
            x2 = int((cx + w/2) * frame_w / 640)
            y2 = int((cy + h/2) * frame_h / 640)
            boxes.append([x1, y1, x2-x1, y2-y1])
            scores.append(float(confidence))
    indices = cv2.dnn.NMSBoxes(boxes, scores, conf_threshold, nms_threshold)
    final_boxes = []
    if len(indices) > 0:
        for i in indices.flatten():
            x, y, w, h = boxes[i]
            final_boxes.append((x, y, x+w, y+h))
    return final_boxes

DELAY = 10
```

```
zone_last_detected = {
    "zone1": 0,
    "zone2": 0,
    "zone3": 0,
    "zone4": 0
}
try:
    while True:
        frame = picam2.capture_array()
        h, w, _ = frame.shape
        input_tensor = preprocess(frame)
        outputs = session.run(None, {input_name: input_tensor})
        boxes = postprocess(outputs, w, h)
        current_time = time.time()
        count_zone1 = 0
        count_zone2 = 0
        count_zone3 = 0
        count_zone4 = 0
        # draw zones
        cv2.line(frame, (w//2, 0), (w//2, h), (255,255,255), 2)
        cv2.line(frame, (0, h//2), (w, h//2), (255,255,255), 2)
        for (x1, y1, x2, y2) in boxes:
            cx = (x1+x2)//2
            cy = (y1+y2)//2
            if cx < w//2 and cy < h//2:
                zone_last_detected["zone1"] = current_time
                count_zone1 += 1
            elif cx >= w//2 and cy < h//2:
```

```
            zone_last_detected["zone2"] = current_time
            count_zone2 += 1
        elif cx < w//2 and cy >= h//2:
            zone_last_detected["zone3"] = current_time
            count_zone3 += 1
        else:
            zone_last_detected["zone4"] = current_time
            count_zone4 += 1
        # draw detection
        cv2.rectangle(frame, (x1,y1), (x2,y2), (0,255,0), 2)
        cv2.circle(frame, (cx,cy), 5, (0,0,255), -1)
    # zone ON/OFF
    zone1 = "ON" if (current_time - zone_last_detected["zone1"]) < DELAY else "OFF"
    zone2 = "ON" if (current_time - zone_last_detected["zone2"]) < DELAY else "OFF"
    zone3 = "ON" if (current_time - zone_last_detected["zone3"]) < DELAY else "OFF"
    zone4 = "ON" if (current_time - zone_last_detected["zone4"]) < DELAY else "OFF"
    total_count = count_zone1 + count_zone2 + count_zone3 + count_zone4
    # GPIO control
    GPIO.output(ZONE_PINS["zone1"], GPIO.LOW if zone1=="ON" else GPIO.HIGH)
    GPIO.output(ZONE_PINS["zone2"], GPIO.LOW if zone2=="ON" else GPIO.HIGH)
    GPIO.output(ZONE_PINS["zone3"], GPIO.LOW if zone3=="ON" else GPIO.HIGH)
    GPIO.output(ZONE_PINS["zone4"], GPIO.LOW if zone4=="ON" else GPIO.HIGH)
    # clear terminal
    print("\033c", end="")
    print("========== ZONE COUNT ==========")
    print(f"Zone 1 : {count_zone1}")
    print(f"Zone 2 : {count_zone2}")
    print(f"Zone 3 : {count_zone3}")
```

```
        print(f"Zone 4 : {count_zone4}")
        print("------------------------------")
        print(f"Total  : {total_count}")
        print("==================================")
        # display counts
        cv2.putText(frame, f"Z1: {count_zone1}", (10, 30),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.7, (0,255,255), 2)
        cv2.putText(frame, f"Z2: {count_zone2}", (w//2+10, 30),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.7, (0,255,255), 2)
        cv2.putText(frame, f"Z3: {count_zone3}", (10, h//2+30),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.7, (0,255,255), 2)
        cv2.putText(frame, f"Z4: {count_zone4}", (w//2+10, h//2+30),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.7, (0,255,255), 2)
        cv2.putText(frame, f"Total: {total_count}", (w//2-70, h-10),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.7, (255,255,0), 2)
        cv2.imshow("Zone Detection", frame)
        if cv2.waitKey(1) & 0xFF == ord('q'):
            break
except KeyboardInterrupt:
    print("Stopped!")
finally:
    picam2.stop()
    cv2.destroyAllWindows()
    for pin in ZONE_PINS.values():
        GPIO.output(pin, GPIO.HIGH)
    GPIO.cleanup()
```


K.Ramakrishnan
College of Technology
Autonomous
Affiliated to Anna University Chennai, Approved by AICTE New Delhi,
Accredited by NBA and with 'A+' grade by NAAC
Samayapuram, Tiruchirappalli - 621 112, Tamilnadu, India.
Incubating Minds,
Catalyzing Careers
IEEE
PEACE
CERTIFICATE
OF PARTICIPATION
This is to certify that SAIRISI N B of RAJALAKSHMI ENGINEERING COLLEGE has authored a paper entitled REAL-TIME INDOOR OCCUPANCY MONITORING WITH IOT AND EDGE COMPUTING and presented in the Second International Conference in Pathway of Electrical, Automation, Communication and Electronics (PEACE-2026) organized by Department of EEE and ECE, K. Ramakrishnan College of Technology (Autonomous), Tiruchirappalli during 27, 28 - March - 2026.
CONVENER/EEE
CONVENER/ECE
HOD/EEE
HOD/ECE


K.Ramakrishnan
College of Technology
Autonomous
Affiliated to Anna University Chennai, Approved by AICTE New Delhi,
Accredited by NBA and with 'A+' grade by NAAC
Samayapuram, Tiruchirappalli - 621 112, Tamilnadu, India.
Incubating Minds,
Catalyzing Careers
IEEE
PEACE
CERTIFICATE
OF PARTICIPATION
This is to certify that SRI MAANESH PA of RAJALAKSHMI ENGINEERING COLLEGE has authored a paper entitled REAL-TIME INDOOR OCCUPANCY MONITORING WITH IOT AND EDGE COMPUTING and presented in the Second International Conference in Pathway of Electrical, Automation, Communication and Electronics (PEACE-2026) organized by Department of EEE and ECE, K. Ramakrishnan College of Technology (Autonomous), Tiruchirappalli during 27, 28 - March - 2026.
CONVENER/EEE
CONVENER/ECE
HOD/EEE
HOD/ECE